

UNIVERSITÉ DE BRETAGNE OCCIDENTALE

FACULTÉ DES SCIENCES ET TECHNIQUES

DÉPARTEMENT INFORMATIQUE

MASTER 2 - SYSTÈMES INTELLIGENTS, INTERACTIFS ET
AUTONOMES

Rapport de Stage

-

Génération de jeu de données pour la compréhension de scènes industrielles

Auteur :

COTTOUR Erell

Encadrants :

BUCHE Cédric

CAZORLA Romain

13 septembre 2023

Remerciements

Je souhaite remercier Cedric BUCHE et Romain CAZORLA pour m'avoir prise dans ce stage, m'avoir aidée pour sa réalisation et avoir été présents malgré la distance qui nous séparait.

Je remercie les membres de l'équipe SMARI de Segula Technologies pour l'accueil et les échanges que nous avons eus.

Je remercie les membres du CERV pour leur accueil chaleureux.

Enfin, je tiens à remercier Pierre CORNEN qui travaillait sur le même projet et qui m'a beaucoup aidée dans ma réflexion, ainsi que les autres stagiaires qui étaient présents au sein du Technopôle.

Abstract

Depuis quelques années, la recherche en compréhension de scènes complexes par des réseaux de neurones s'est grandement développée. Afin de permettre au réseau d'apprendre, il faut lui mettre à disposition des jeux de données qui soient complets, qui correspondent aux besoins de l'utilisateur et soient correctement annotés. De plus en plus de jeux de données se sont développés ces dernières années, et la recherche se poursuit dans ce domaine. Cependant, ces jeux de données se concentrent principalement sur des scènes d'intérieur et quelques-uns sur des scènes extérieures. Actuellement, il y a très peu de jeux de données industriels qualitatifs, ainsi que de générateurs de scènes permettant de créer tous types de scènes. Notre objectif est de permettre à un ordinateur d'identifier chaque élément d'une scène industrielle récupérée via un capteur laser. Pour cela, nous fournissons des nuages de points pour l'apprentissage en Deep Learning.

Il s'agit du sujet sur lequel nous avons travaillé ces cinq derniers mois. Ce papier décrit l'analyse du générateur de maisons ProcTHOR [1] qui a été effectuée, ainsi que le générateur de scènes 3D qui a été créé durant cette période. Les scènes 3D sont par la suite transformées en nuages de points. Nos problématiques étaient le besoin d'adaptabilité du code pour plusieurs types de scènes et plusieurs jeux de données et le réalisme des scènes générées, notamment avec le positionnement des objets.

Table des matières

1	Introduction	4
2	Résumé de la bibliographie	5
2.1	Capture de données	5
2.2	Annotation des scènes	6
2.3	Qualité des données	8
2.4	Bilan	11
2.5	Conclusion	12
3	Méthode choisie	13
3.1	Méthodologie	14
3.2	Matériel utilisé	16
4	Analyse de ProcTHOR	16
4.1	Fonctionnement global de ProcTHOR	16
4.2	Après modification des variables	17
5	Réalisation d'un générateur	21
5.1	Création et exportation de la scène	22
5.2	Importation et placement des meubles	23
5.3	Implémentation des objets sur les meubles	27
5.4	Arbre AABB pour créer les BoxColliders	28
5.5	Conclusion	30
6	Résultats	31
7	Conclusion	32
	Références	34
8	Annexes	37
8.1	Installation de l'environnement de travail pour ProcTHOR [1]	37
8.2	Structure de ProcTHOR [1]	38
8.3	Modification des variables de ProcTHOR [1]	40
8.4	Générations de scènes avec des objets sur les meubles	44
8.5	Bibliographie	44
8.5.1	Abstract	44
8.5.2	Introduction	45
8.5.3	Capture de données	46
8.5.4	Annotations	52

8.5.5	Qualité des données	58
8.5.6	Bilan	64
8.5.7	Conclusion	68

1 Introduction

Le projet sur lequel s'est déroulé ce stage est le projet SMARI de SEGULA Technologies. Il a pour objectif d'aider les ingénieurs dans la maintenance de structures industrielles. Cela passe notamment par la modélisation CAD de ces environnements, ce qui représente un travail long et minutieux. Ainsi, le projet SMARI consiste à scanner les zones industrielles sous la forme de nuages de points. Puis les nuages de points sont segmentés afin de générer des modèles CAD du site industriel.

Cela fait quelques années que les chercheurs ont commencé à s'intéresser à la compréhension de scènes d'intérieur (scènes domestiques) et d'extérieur (scènes urbaines) complexes. Ce sont des scènes composées de nombreux objets, et qui appartiennent au quotidien. Il existe de nombreuses scènes qui ont peu été traitées par la littérature, notamment les scènes industrielles. Ce sont des scènes qui peuvent facilement être encombrées, avec un risque élevé d'obstruction sur de nombreux éléments. L'objectif est de pouvoir identifier chaque élément et de pouvoir se repérer dans l'espace.

En réalité virtuelle, l'appréhension de l'environnement dans lequel évolue l'utilisateur est devenu un point très important. L'utilisation du Deep Learning, ou apprentissage profond, permet d'arriver à de bons résultats pour la compréhension de scènes complexes. Cependant, il faut avoir de bons jeux de données pour l'apprentissage, en bonne quantité. Aujourd'hui, peu de jeux de données sont vraiment performants, notamment en milieu industriel.

Les scènes peuvent exister sous différentes formes. Ici, ce sont des nuages de points qui nous intéressent pour l'apprentissage, donc des données de profondeur. En plus des données de profondeurs, nous récupérons les données des couleurs des objets. Ce sont des données RGB-D (Red Blue Green Depth). Certains jeux de données sont constitués de ces données, comme SceneNet-RGBD [2] ou OpenRooms [3].

Cependant, il est aussi possible de récupérer des jeux de données constitués de scènes 3D comme ProcTHOR [1] afin de les modifier en nuages de points pour ensuite les utiliser dans l'apprentissage. Pour ProcTHOR [1], les données sont intégralement synthétiques, c'est-à-dire qu'elles ont été modélisées et créées par ordinateur. D'autres sont récupérées dans le monde réel à partir de capteurs tels que des appareils photo ou des lasers de profondeur. D'autres données encore ont été récupérées à partir de capteurs virtuels déposés dans des jeux vidéos.

Pour qu'un ordinateur apprenne en Deep Learning, il lui faut beaucoup de données pour s'entraîner. De nombreux jeux de données sont apparus, mais aucun ne correspond aux besoins du projet, soit un jeu de données industrielles suffisamment précis. Nous devons donc trouver un moyen de le créer.

L'objectif de ce stage était donc de créer un générateur de scènes industrielles. Il se passe au sein du CERV (Centre Européen de Réalité Virtuelle) et est supervisé par Cédric BUCHE et Romain CAZORLA. Le stage se positionne dans le cadre du doctorat de Romain CAZORLA et s'étendait sur cinq mois. La détection de trous dans des nuages de points et leur complétion est une autre partie du projet SMARI et a été réalisée par un autre stagiaire, Pierre CORNEN, sur la même période.

2 Résumé de la bibliographie

Une étude bibliographique a été effectuée en amont du stage sur les différentes techniques de création de jeux de données qui servent à l'apprentissage profond pour la compréhension de scènes complexes. Cette étude bibliographique présente différents jeux de données avec, dans un premier temps, les méthodes de capture de données, puis les méthodes de créations pour l'annotation des scènes et enfin la qualité des données fournies dans les différents jeux de données et un bilan sur ce qui pourrait nous intéresser dans le cadre du stage. La bibliographie est disponible à l'annexe 8.5.

2.1 Capture de données

L'acquisition est une étape importante pour la constitution d'un jeu de données. Celui-ci doit être assez conséquent avec des données de bonne qualité qui doivent être variées pour que l'apprentissage se passe correctement. Il existe plusieurs méthodes pour les capturer.

Données du monde réel

Afin de capturer des données du monde réel, plusieurs appareils peuvent être utilisés, comme des caméras, qui capturent des données RGB. Il y a des capteurs de profondeur (capteurs LIDAR) qui sont des lasers qui génèrent un nuage de points de la scène et des caméras RGB-D (Red, Green, Blue, Depth) qui récupèrent des images et créent une carte de profondeur. Cette méthode de récupération permet d'avoir des données réalistes, ce qui est important pour l'apprentissage. Cependant, elles ont un fort coût humain, notamment pour le temps de capture avec l'appareil à configurer. Cela limite souvent la quantité de données présente dans le jeu.

Données synthétiques

Il est possible de générer des données par ordinateur pour créer un jeu de données synthétique. Pour les réaliser, il faut récupérer des données afin que les scènes soient les plus cohérentes possible. Il est nécessaire de s'approcher de la réalité physique et la cohérence par rapport au monde réel pour un meilleur apprentissage. Certains jeux de données, comme SceneNet RGBD [2], ont des scènes chargées avec de la cohérence entre les meubles posés et la pièce, mais dont les meubles sont disposés aléatoirement dans l'espace. Un autre jeu de données appelé ProcTHOR, créé par Deitke et coll. [1], est réalisé à partir d'un générateur de scènes d'intérieur complexes avec une cohérence dans la disposition des meubles. Dans les deux cas, le nombre de données est plus conséquent que les captures réalisées dans le monde réel. Les données synthétisées ont l'avantage d'avoir un coût humain plus léger et la génération de scènes plus rapide que la capture. Cependant, il est difficile d'obtenir des scènes réalistes, ce qui peut nuire à l'apprentissage.

Bilan de la partie

La quantité de données présentes dans un jeu de données est importante pour l'apprentissage d'un réseau de neurones en Deep Learning, ainsi que la qualité des données. Les données doivent être relativement variées pour un bon apprentissage. Ces critères seront impactés par la méthode de capture ou de création de données. De plus, plus l'humain doit intervenir dans le processus, plus il faudra de temps pour créer le jeu de données, ce qui impacte la quantité. Cependant, les données seront aussi plus réalistes. Nous pouvons voir avec ce tableau 1 que les jeux de données qui ont des données générées automatiquement [2, 1] ont plus de données annotées que tous les autres. Il faut tout de même noter qu'il est difficile de comparer la performance des différents jeux de données, tout d'abord parce qu'ils ne sont pas tous sur les mêmes types de données, et qu'il est difficile de trouver des articles comparant les différents jeux de données entre eux.

2.2 Annotation des scènes

Pour qu'un réseau de neurones puisse apprendre à partir des données qui lui sont fournies, il est important que ces dernières soient bien annotées. ShapeNet [9] possède trois types d'annotations : les annotations géométriques qui organisent les formes par catégories de propriétés géométriques, les annotations fonctionnelles qui décrivent de fonctionnement d'objets pouvant changer d'état et les annotations physiques qui correspondent notamment à

Jeu de données	Nombre d'agence- ment de scènes	Nombre de configurations	Nombres de scènes annotées
SceneNet	57	16895	5M
RGBD [2]			
OpenRooms [3]	-	100 000	-
ScanNet [4]	1513	1513	2,5M
Scan2CAD [5]	3049	14 225	-
SceneNN [6]	100	100	-
ArkitScenes[7]	5047	5047	450 000
SqueezeSeg [8]	-	10 848	-
ProcTHOR [1]	18	1633	100Md
ShapeNet [9]	-	3M	220 000

TABLE 1 – Comparaison des différents jeux de données selon leur quantité de données

la dimension et la densité des objets.

La plupart du temps, les annotations se font pixel par pixel où sont attribuées une ou plusieurs valeurs correspondant à l'ensemble auquel ils appartiennent, des données de profondeurs s'il y en a, et toute autre donnée qui peut être importante pour le jeu de données. Il existe différents types d'annotations. Pour l'identification d'objets, les annotations auront tendance à être des boîtes qui englobent le contour des objets, tandis que pour une compréhension plus générale d'une scène, l'annotation aura plus tendance à se faire pixel par pixel. Certains jeux de données possèdent aussi plusieurs couches d'annotations, comme SceneNN [6] par Binh-Son Hua et coll. qui ont créé des annotations fines qui correspondent aux contours de l'objet, et grossières qui sont la fusion des segments fins. Puis un utilisateur vient affiner l'étiquetage à la main.

Étiquetage manuel

L'étiquetage manuel est le plus long et celui qui est le plus propice aux petites erreurs. Cela est vrai avec l'étiquetage par pixel, mais aussi selon le type de données, comme les données industrielles qui demandent plus de travail. Il existe des méthodes d'étiquetage mixtes qui demandent à l'utilisateur d'assister l'ordinateur qui effectue une partie du travail, ce qui allège la charge de l'utilisateur. Pour SceneNN [6], les deux premières couches sont créées automatiquement via l'ordinateur, puis un utilisateur vient affiner le tout. Pour ArkitScenes [7], il y a un outil pour annoter les boîtes englobantes créées avec une projection en temps réel des délimitations de boîtes afin de faciliter la démarche.

Étiquetage automatique

Un étiquetage automatique se fait bien plus rapidement, et est souvent bien plus performant et moins source d'erreurs. Cependant, il faut que la scène soit assez nette et bien compréhensible pour l'ordinateur afin qu'il ne fasse pas d'erreurs, sachant qu'elles peuvent vite être conséquentes. Il est difficile d'étiqueter sans avoir à passer par le Deep Learning, ce qui signifie qu'il faut déjà avoir des données pour permettre l'apprentissage. Ankur Handa et coll. [10] ont utilisé une architecture d'encodeur-décodeur construite sur le réseau VGG. Le réseau VGG récupère les données RGB de chaque pixel, affine et diminue la taille de l'image afin de ne conserver que les données discriminantes, afin de permettre au réseau de neurones de déterminer le contour des objets et surfaces de la scène. Puis les objets sont identifiés et annotés selon leur forme.

Bilan de la partie

La qualité d'un jeu de données dépend de son étiquetage. Il s'agit de la partie qui demande le plus de temps à réaliser. Cela peut limiter la quantité de données présentes dans le jeu. Les étiquetages manuels demandent plus de main d'œuvre, de temps de travail et peuvent être à l'origine de plus de petites erreurs. Les annotations automatiques peuvent aisément retrouver la forme des objets, mais les différentes annotations demandent déjà une compréhension de la scène, donc un travail assez conséquent en amont et peuvent être source d'erreurs plus conséquentes. Une annotation hybride est, actuellement, une bonne alternative pour les scènes capturées du monde réel. Les scènes générées peuvent, elles, plus aisément annoter chaque élément de la scène. Le tableau 2 montre les différentes méthodes de capture de données, ainsi que le mode d'étiquetage lié.

2.3 Qualité des données

Afin que les données fournies soient utiles pour l'apprentissage, et correspondent au mieux au monde réel, il existe plusieurs variables sur lesquelles nous pouvons jouer : la luminosité, les différentes vues, les matériaux utilisés, le bruit et la variété des données. Tous ces éléments permettent de juger, au moins en partie, de la qualité d'un jeu de données.

Vues de la caméra

Lorsque les données utilisées viennent du monde réel, le nombre d'axes de vue possible est en général limité, d'autant plus si les données sont récu-

Jeu de données	Méthode de capture des données	Méthode d'étiquetage
SceneNet RGBD [2]	Génération automatique	Automatique
OpenRooms [3]	Reprise jeu Scan2CAD	Automatique
ScanNet [4]	Caméras RGB-D	Automatique
Scan2CAD [5]	Caméras RGB-D	Manuelle
SceneNN [6]	Caméras RGB-D	Hybride
ArkitScenes[7]	Caméras RGB	Manuelle (envi-
	Capteurs LIDAR	ronnement amélioré)
AdobeIndoorNav [11]	Caméras RGB-D	-
SqueezeSeg [8]	Reprise jeu KITTI	Automatique
	Données du jeu GTA V	
ProcTHOR [1]	Génération automatique	-
ShapeNet [9]	Reprise jeux de données	Hybride

TABLE 2 – Comparaison des différents jeux de données selon leur méthode de capture et d'étiquetage

pérées d'un jeu déjà existant. Selon le point de vue, les données seront plus ou moins intéressantes pour l'apprentissage. Une voiture autonome aura des capteurs à une même position, il faudra donc que les données fournies pour l'apprentissage correspondent à leur hauteur.

La modélisation en 3D d'objets permet de les capturer sous tous les angles voulus. Ankur Handa et coll. [10] modélisent les scènes initialement récupérées avec des caméras et capteurs de profondeur, puis créent des images avec un point de vue aléatoire afin de permettre au réseau de neurones de s'entraîner selon tous les angles de vue. Cependant, ce travail a un coût étant donné qu'il faut, en plus de la capture initiale, transformer une image 2D en scène 3D, ce qui demande du temps.

Luminosité

La luminosité et la position du soleil évoluent en fonction de l'heure de la journée, des saisons et de la météo et peuvent créer de forts contrastes (par exemple sur les couleurs). De même, plusieurs appareils électriques émettent une lumière plus ou moins fort avec une coloration variée. Matt Deitke et coll. [1] ont travaillé sur la lumière d'intérieur. En effet, chacune de leurs scènes est intégralement interactive, avec la possibilité de modifier l'état de nombreux objets, dont les lumières. Ainsi, il est possible de faire varier la luminosité d'une lampe, ainsi que la couleur de son ampoule.

Matériaux

Les matériaux ont un fort impact sur la scène, il est important de pouvoir les modéliser. Lors de la récupération de données du monde réel, le problème ne se pose pas, mais avec des modélisations de scènes en 3D, cela devient plus important. En effet, un miroir, du cuir et du bois ne reflèteront pas la lumière de la même manière, ce qui peut être impactant pour l'apprentissage. Il en va de même pour les textures. La version la plus basique des matériaux sont de simples couleurs.

Certains jeux comme OpenRooms [3] de Zhengqin Li et coll. et ProcTHOR [1] de Matt Deitke et coll. utilisent des matériaux de plusieurs catégories (par exemple : le tissu, le cuir, le métal, le plastique) pour améliorer le réalisme.

Bruit

Les données qui sont capturées du monde réel ont systématiquement du bruit. Les capteurs de profondeur possèdent beaucoup de points qui leur seront obstrués par d'autres objets. Ce bruit aura donc un impact pour l'apprentissage et la compréhension de scènes par la suite. Le jeu de données doit correspondre aux données auquel l'ordinateur sera confronté après apprentissage. Miho Adachi et coll. [12] parlent de la nécessité de faire comprendre à un robot qui se balade en extérieur et qui utilise des données de profondeur que l'immense trou au-dessus de lui est le ciel. Lorsque les données sont générées automatiquement, elles sont trop parfaites, il convient donc d'ajouter du bruit dans le jeu de données.

À l'inverse, les données du monde réel auront tendance à avoir beaucoup de bruit. Or, s'il y a trop, l'apprentissage sera mauvais. Il faut donc parfois les atténuer, par exemple en faisant correspondre différents points de vue.

Types de données

Selon les besoins pour l'apprentissage, il n'y aura pas forcément besoin de connaître l'intégralité des objets pouvant exister, alourdissant inutilement les jeux de données, et complexifiant l'apprentissage.

Matt Deitke et coll. pour ProcTHOR [1] ont créé un jeu de données uniquement basé sur le domicile avec la génération de 10 000 maisons. Il y aura donc un manque si on souhaite analyser une scène dans un magasin ainsi qu'une scène en extérieur.

De leur côté, Ekta U. Samani et coll. [13] se sont intéressés à la modélisation d'objets afin de pouvoir les identifier dans des scènes encombrées. Ils ont donc beaucoup de petits objets dans leur jeu de données.

Bilan de la partie

La qualité des données fournies par le jeu de données est importante pour un bon apprentissage. Il faut qu'elles soient suffisamment proches de la réalité pour que les données puissent être le plus générique possible. Dans les tableaux 3 et 4, nous pouvons voir le résumé des différents axes, explicités plus haut, représentant la qualité des différents jeux de données. Nous pouvons constater que peu de jeux de données travaillent avec les matériaux, la lumière et le nombre de vues, et que la majorité d'entre eux se contentent de données RGB-D.

Jeu de données	Type de données	Type de scène	Lumière	Matériaux
SceneNetRGBD[2]	RGB-D	Habitation	Statique	Couleurs
OpenRooms[3]	RGB-D	Habitation	Variée	Nombreux
ScanNet[4]	RGB-D	Intérieurs	Statique	Couleurs
Scan2CAD[5]	RGB-D	Habitation	Statique	Couleurs
SceneNN [6]	Thermique RGB-D	Habitation	Ré-éclairage possible	Couleurs
ArkitScene[7]	RGB-D	Habitation	Statique	Couleurs
AdobeIndoorNav[11]	Profondeur	Habitation	Statique	-
SqueezeSeg[8]	RGB	Extérieur	-	-
ProcTHOR[1]	RGB	Habitation	-	Nombreux
ShapeNet[9]	RGB-D	Objets	Couleurs	Vues limitées

TABLE 3 – Comparaison des différents jeux de données selon les caractéristiques de leurs données

2.4 Bilan

Il est difficile de comparer l'efficacité des différents jeux de données créés, sachant que tous les jeux de données ne se concentrent pas sur les mêmes aspects, et qu'il n'y a, actuellement à ma connaissance, pas de comparaisons réalisées entre les différents jeux de données selon les catégories. De plus, beaucoup de jeux de données actuels sont principalement axés sur les scènes d'intérieur et plus précisément les domestiques. Les propositions restent assez limitées, malgré un réel besoin dans l'industrie et dans de nombreux autres domaines. Il en va de même pour les données proposées en sortie. La majorité des jeux de données sont RGB-D, donc principalement des images avec

Jeu de données	Points de vue	Réalisme de la scène	Bruit
SceneNet RGBD [2]	Vues limitées	Peu réaliste	Aucun
OpenRooms [3]	Vues limitées	Réaliste	Aucun
ScanNet [4]	Vues limitées	Réaliste	Par obstruction
Scan2CAD [5]	Vues limitées	Réaliste	Aucun
SceneNN [6]	Vues limitées	Réaliste	Par obstruction
ArkitScenes[7]	Vues limitées	Réaliste	Par obstruction (très peu)
AdobeIndoorNav [11]	Nombreuses trajectoires	Réaliste	Réaliste
SqueezeSeg [8]	-	Réaliste	Par obstruction
ProcTHOR [1]	-	Réaliste	Aucun
ShapeNet [9]	-	Réaliste	Aucun

TABLE 4 – Comparaison des différents jeux de données selon les caractéristiques de leurs données

quelques informations sur la profondeur, et peu de nuages de points ou de données thermiques par exemple.

L’annotation est une étape primordiale et chronophage pour la création d’un jeu de données. Si elle doit être faite à la main, le temps passé dessus est d’autant plus conséquent, et est source de nombreuses petites erreurs qui peuvent gêner pour l’apprentissage. À l’inverse, des annotations automatiques ont besoin d’un algorithme suffisamment performant pour pouvoir correctement annoter les scènes. Cette manière de faire est bien plus rapide, mais dont les erreurs seront bien plus conséquentes.

Enfin, nous pouvons aussi constater que la littérature s’intéresse peu aux nuages de points, et que la question de la transformation du maillage d’une scène en nuage de points n’est pas traitée. Cette question pourrait pourtant énormément apporter lors de l’apprentissage d’un réseau de neurones.

Cependant, certains jeux de données montrent des résultats plutôt encourageants, comme Matt Deitke et coll. avec ProcTHOR [1] qui ont de très bons résultats dans la génération de scènes d’intérieur domestiques, avec de nombreux objets, matériaux, luminosités et angles de caméras possibles, et dont les scènes sont réalistes physiquement, et cohérentes.

2.5 Conclusion

Cette étude bibliographique présente plusieurs jeux de données variés pour la compréhension de scènes complexes, que ce soit en intérieur ou en

extérieur.

Les points discriminants entre les jeux les plus importants que nous avons pu trouver sont indiqués sur la figure 1. Il s'agit de la qualité des données, soit, le travail sur l'éclairage, les matériaux, le bruit et les vues de la caméra. Il y a aussi le type d'annotations et leur nombre de couches, le type de données dont est composé le jeu, et le réalisme des scènes proposées. Nous avons pu voir de bons résultats dans la création de jeux de données performants dans la compréhension de scènes d'intérieur domestiques.

Il serait donc intéressant, dans l'objectif de créer notre propre générateur de données, de reprendre un générateur ayant de bons résultats pour l'adapter aux données industrielles. Nous pouvons citer ProcTHOR [1] de Matt Deitke et coll. qui a de bonnes performances pour la génération de maisons.

Capture des données	Annotations	Qualité des données
Données du monde réel	Manuelles	Vues de la caméra
- Plus lent - Plus grand coût humain - Moins de données - Réaliste	- Très lent - Grand coût humain - Plus propice aux erreurs	
Données générées	Automatiques	Travail sur la lumière
- Coût humain faible - Plus rapide - Grande quantité de données - Pas toujours réaliste	- Coût humain faible - Rapide - Erreurs sont plus conséquentes	Travail sur les matériaux
Reprise de bases de données	Mixte	Bruit
- Plus rapide - Plus grand coût humain	- Allège de coût humain - Diminue les erreurs	Types de données
Capture d'un jeu vidéo		
- Plus rapide - Plus grande quantité de données - Pas toujours fiable		

FIGURE 1 – Schéma résumant les conclusions de l'étude bibliographique.

3 Méthode choisie

Afin de créer un jeu de données pour un apprentissage de compréhension de scènes, nous avons travaillé selon plusieurs étapes. L'objectif étant à la fois de réussir à créer un générateur de scènes qui soit facilement modulable,

mais aussi de confirmer que le réalisme d'une scène peut avoir un impact sur l'apprentissage.

3.1 Méthodologie

Afin de répondre à la problématique, nous avons travaillé sur un générateur de scènes qui permettrait de créer un jeu de données pour l'apprentissage de la compréhension de scènes industrielles. La méthodologie de travail qui a été employée est visible sur la figure 2.

L'idée initiale est de reprendre un générateur de scènes d'intérieur déjà existant afin de l'adapter pour en faire un générateur de scènes industrielles. Nous avons donc décidé de reprendre ProcTHOR [1], un générateur de maisons, étant donné qu'il a de bons résultats afin de réaliser un générateur de scènes 3D. Le projet initial est de générer des scènes en 3D à partir du générateur repris tout en adaptant les scènes selon nos besoins. Ensuite, pour avoir le rendu le plus réaliste possible, nous simulons un laser qui suit les règles de la physique afin de générer un nuage de points de la scène 3D. Cela permet d'obtenir des rendus proches de la réalité, avec les mêmes défauts que si un capteur avait été déposé dans le monde réel. Même si ce dernier est gourmand en ressources, il est nécessaire pour un bon apprentissage avec le jeu de données généré. Ces nuages de points sont ensuite utilisés dans un algorithme d'apprentissage en Deep Learning. Le générateur créé va être testé avant d'être adapté pour des scènes industrielles. Il doit respecter plusieurs critères :

- Pouvoir réaliser des scènes variées selon les besoins de l'utilisateur. Cela passe par la possibilité de donner le type de pièce voulu par l'utilisateur (par exemple un bureau, une cuisine).
- Avoir un accès simple aux variables pour permettre de modifier facilement les scènes générées. Les variables concernées sont la taille du bâtiment, le nombre moyen d'objets par pièce, la quantité et/ou densité d'objets et de meubles dans les pièces et le type d'intérieur voulu.
- Utiliser des objets et meubles provenant d'un jeu de données déjà existant, avec la possibilité d'importer ceux de son choix.
- Réaliser une répartition des objets physiquement possible et cohérente.
- Avoir une bonne gestion des textures et de la lumière.
- Avoir des formes de pièces variées.

Afin de réaliser le générateur, dans un premier temps, une évaluation qualitative de ProcTHOR [1] est effectuée. Elle est représentée dans la partie de gauche du schéma 2. Puis un générateur est réalisé, soit à partir des bases de ProcTHOR [1] si les résultats sont corrects, soit en créant un nouveau

générateur ne se basant pas sur le code de Dans un premier temps, une analyse du fonctionnement de ProcTHOR [1]. Cela correspond à la partie droite du schéma 2.

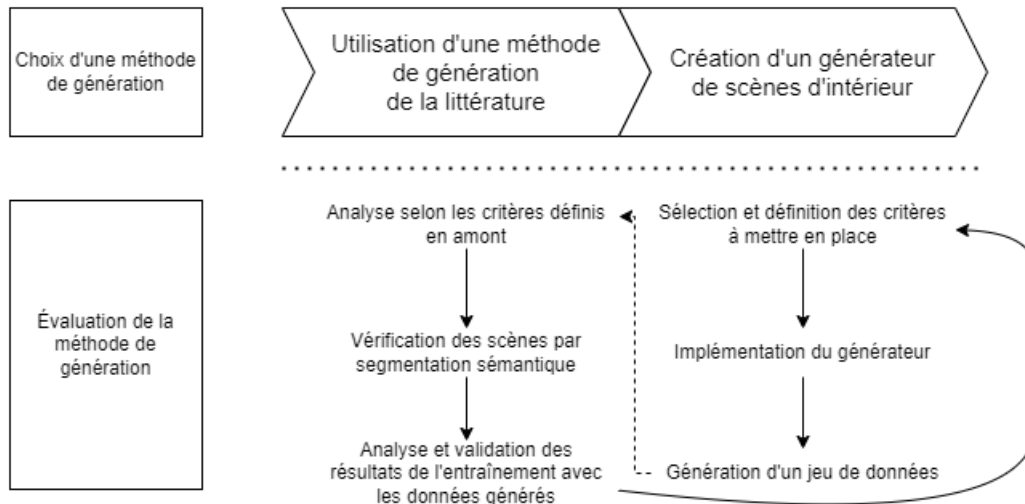


FIGURE 2 – Méthodologie utilisée lors du stage.

Méthode de vérification par segmentation sémantique

Par la suite, on pourra vérifier que les scènes sont correctes via segmentation sémantique. La méthode de segmentation est KP-FCNN avec un kernel non déformable [14]. Quarante scènes provenant du générateur créé seront transformées en nuages de points puis ajoutées à la base de données S3DIS qui est déjà utilisée dans le projet SMARI. Le jeu de données possède six zones, la 5 étant pour les tests, et les autres pour l'entraînement. Lors de l'apprentissage, 16 scènes aléatoires sont sélectionnées et un pilier d'un mètre sur un mètre créé d'un point aléatoire dans la scène est utilisé. Puis, une rotation aléatoire est appliquée au niveau de l'axe vertical pour chaque pilier. L'entraînement se fait sur 300 epochs comportant chacune 1000 nuages de points. Dans un premier temps, un apprentissage sans les données du générateur sera effectué afin de pouvoir comparer l'évolution des résultats avec le générateur.

Les nuages de points de données synthétiques sortiront de couleur blanche, tandis que les données provenant du monde réel conserveront leurs couleurs. Un bruit Guassien léger est ajouté à la position des normales des points données en entrée (moyenne 1 cm, variance 0 et bruit max 5 cm). Chaque nuage généré possède 4096 points qui sont répartis de manière équitable dans l'espace de la scène (Farthest Point Sampling). La première couche de pooling

utilise un rayon d'échantillonnage de 4 cm. Adam est utilisé comme optimizer avec un taux d'apprentissage de base de 0.001. La taille de batch est de 16.

3.2 Matériel utilisé

Lors de mon stage, j'ai travaillé sur un ordinateur portable MSI sous Windows 10 avec un processeur Intel Core I7 8^e génération et une carte graphique Nvidia GTX 1070.

Les générations sous ProcTHOR [1] ont été faites sur un environnement Anaconda, en utilisant Cuda 11.3 avec une version de PyTorch 1.12.1, sous WSL 2. Le générateur a été réalisé sur le logiciel Unity version 2021.3.23f1 en C#.

4 Analyse de ProcTHOR

Dans un premier temps, une analyse du fonctionnement de ProcTHOR [1] ainsi que ses résultats sont effectués. L'installation de ProcTHOR [1] est décrite dans la section 8.1.

4.1 Fonctionnement global de ProcTHOR

Afin d'observer les résultats des générations de ProcTHOR [1] et pouvoir constater leur efficacité, plusieurs générations de maisons ont été effectuées. De plus, pour se familiariser avec son code et observer son déroulé, nous avons utilisé le débogueur de Visual Studio Code. Le schéma en figure 17 de l'annexe montre la structure globale du code de ProcTHOR [1]. Il est clairement visible que ProcTHOR [1] se divise en deux parties, la partie AI2-THOR qui est un jeu de données constitué de 125 types d'objets différents avec plusieurs types de textures applicables pour chaque objet. AI2-THOR représente la partie bleue du schéma 17 en annexes. Ce jeu de données a été créé par la même équipe que celle qui a réalisé ProcTHOR [1]. Il sert à meubler les plans créés par ProcTHOR [1]. La seconde partie, visible en orange sur le schéma 17, permet de générer le plan des maisons et créer les scènes 3D, le tout s'articulant autour de la classe *House*. Cette classe est la base de la structure du code, appelant toutes les autres classes afin de générer les plans et l'environnement 3D.

Aspect global des générations sans toucher aux paramètres

Lors de la génération de maisons, sans toucher au code de ProcTHOR [1], plusieurs points positifs cités dans l'article présentant le générateur [1] sont observés lors des générations réalisées. Tout d'abord, les scènes générées sont bien physiquement possibles, mais aussi cohérentes. Cela est visible sur les figures 3 et 4 où nous pouvons voir que les meubles sont tous dans le bon sens et disposés de manière cohérente par rapport à ce qui pourrait être fait dans une maison classique. Il y a sur la figure 3 des captures d'écran faites dans la maison de pièces choisies aléatoirement et sur la figure 4 des plans de maisons générées.

La figure 3 montre aussi qu'il y a une bonne gestion des textures et des matériaux dans les différentes scènes avec des textures variées et plutôt cohérentes. Par exemple, des textures comme du bois, un miroir, du métal ou de la roche ne réagissent pas pareil avec la lumière. Cependant, la gestion de la lumière semble moins satisfaisante, notamment au niveau des ombres qui sont très peu visibles, nous pouvons notamment le constater sur la figure 3 au niveau des différents meubles qui ont peu, voire aucune ombre au sol.

Nous pouvons aussi noter que les pièces sont en général relativement vides, sans aucun désordre. La figure 3, et les plans des maisons de la figure 4 le montrent bien. En effet, au-delà d'être peu meublées, les tables et autres grands meubles ont peu d'objets déposés sur eux, et quasiment rien qui soit déposé au sol. Les rares objets que nous avons pu constater au sol sont des sacs-poubelle ou des plantes.

Le premier aperçu de ProcTHOR [1] est légèrement mitigé, les plans obtenus étant plutôt réalistes et cohérents, mais restant malgré tout trop vide et trop "rangés" par rapport à un environnement dans lequel vivent des humains. Il n'y a pas de notion de pièce encombrée, ce qui pourrait nuire à l'apprentissage. À l'inverse, nous pouvons citer SceneNet-RGBD [2] qui crée des scènes non cohérentes, mais très encombrées. L'objectif suivant est d'essayer de jouer avec les variables de ProcTHOR pour voir s'il est possible de créer des environnements plus complexes.

4.2 Après modification des variables

L'étape suivante fût la modification des variables de ProcTHOR [1]. La liste des variables testées est sur le tableau 5. Nous n'allons présenter qu'une partie d'entre elles. Le premier constat est que les scènes sont peu variables, et que nous obtenons rapidement des résultats similaires en effectuant plusieurs générations, ce qui est perceptible sur la figure 5. Ce n'est pas l'idéal

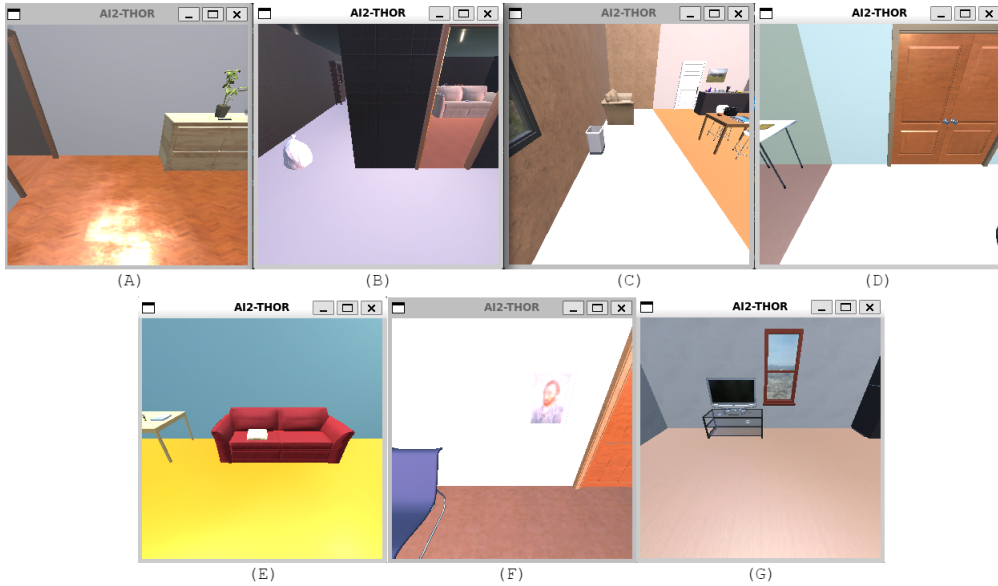


FIGURE 3 – Images de maisons générées par ProcTHOR [1].

pour un algorithme d'apprentissage d'avoir des cas trop proches les uns des autres, cela pourrait engendrer un sur-apprentissage de ce jeu de données. Il aurait donc de bons résultats, mais uniquement pour des cas précis, et serait incapable d'analyser d'autres types de données.

Gestion des grands espaces

Les scènes industrielles étant souvent de grande taille, l'idéal serait de pouvoir réaliser des pièces plus grandes que celles proposées par défaut par ProcTHOR [1]. Il est possible d'agrandir la taille moyenne des pièces avec la variable globale `AVERAGE_ROOM_SIZE`. Plusieurs générations ont été effectuées en la faisant varier, sachant que les scènes industrielles sont en général composées de pièces de grande taille. Le résultat est visible sur la figure 6 et il démontre que ProcTHOR ne parvient pas à gérer les grands espaces. En effet, plus les espaces sont grands, moins la gestion de leur forme est cohérente, c'est notamment perceptible sur la génération *C* sur la figure 6. De plus, la gestion des textures se dégrade aussi, avec de moins en moins de textures variées proposées pour les murs et le sol, allant jusqu'à une unique texture choisie sur la génération *D* de la figure 6. La répartition des objets devient aussi de plus en plus hasardeuse, avec un nombre d'objets présents dans la scène très faible par rapport à sa taille. Plus l'espace est grand, moins les pièces ont de cohérence.

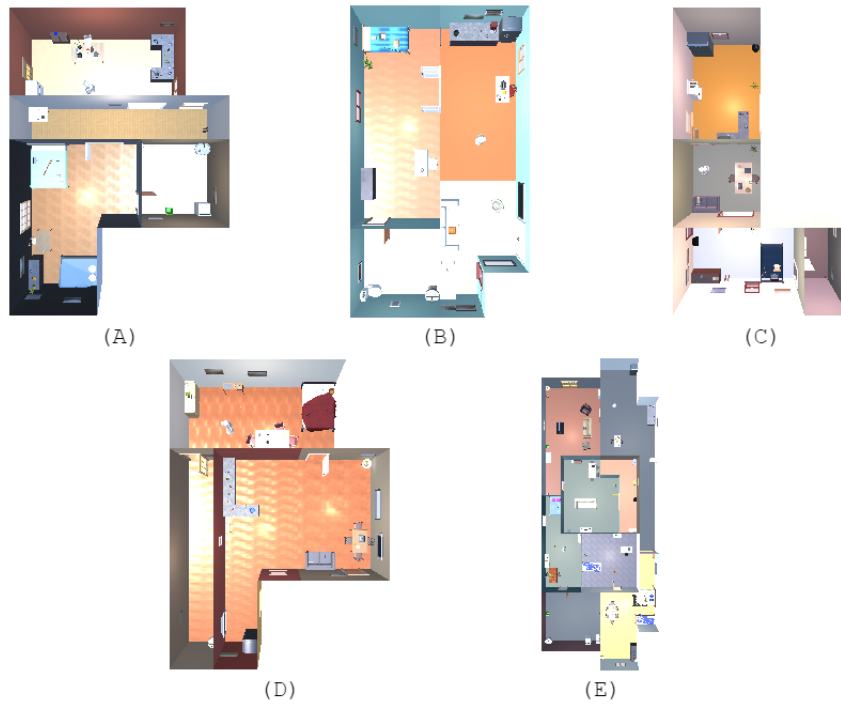


FIGURE 4 – Plans de maisons générées par ProcTHOR [1].

Fiabilité et modification des variables

Un autre problème qui a été relevé est la fiabilité des variables. En effet, malgré la modification de la valeur de certaines variables, il n’y a régulièrement aucun changement de visible sur les générations. Tout d’abord, il est possible de constater qu’en utilisant une même graine aléatoire, nous n’obtenons pas exactement la même maison à chaque génération (cf. figure 7). La forme de la maison se maintient, mais le nombre de pièces, leur forme, les textures et la disposition des objets et meubles varient. Cela signifie qu’il n’est pas possible de générer deux fois la même maison à partir du même code. De même, la variable `num_rooms` qui est décrite comme le nombre de pièces dans une maison, n’impacte pas le nombre de pièces dans la maison, mais la forme des maisons générées. Ce qui est visible en annexes sur la figure 18.

De la même manière, nous pouvons voir que modifier certaines variables n’impacte pas la génération d’une scène. Par exemple, la variable `P_ALLOW_HOUSE_PLANT_GROUP` qui permet de faire varier le nombre de plantes dans la scène ne change rien comme le montre la figure 8. Cela est le cas pour de nombreuses variables comme le nombre de pièces, le nombre de téléviseurs

Nom variable	Description	Effet
seed	Graine pour l'aléatoire	La même ne génère pas la même maison
room_specs	Dimensions en longueur et largeur de la maison	Aucun changement
AVERAGE_ROOM_SIZE	Taille des pièces moyenne	Modifie la taille des pièces
num_rooms	Nombre de pièces dans une maison	Modifie la forme des maisons
PROCTHOR10K_ROOM_SPEC_SAMPLER	Types de pièces possibles dans la génération	Aucun changement
material_database (fichier)	Toutes les textures possibles par catégorie	Limite les textures possibles à celles présentes dans le fichier (cf. figure 20 en annexes)

TABLE 5 – Description des variables et impact sur les générations lorsqu'elles sont modifiées.

moyen, ou la variable globale `MIN_HOUSE_SIDE_LENGTH` qui définit la taille des murs des maisons en longueur. Pour générer les dimensions x et y de la maison, la variable `dims` de `room_specs` utilise aussi l'aléatoire. Dans ce cas, les données sont aussi générées aléatoirement, entre deux valeurs définies et fixes. Plusieurs générations ont été effectuées et sont visibles en annexes sur la figure 19.

La variable `room_spec` récupère les données des pièces via la variable globale `PROCTHOR10K_ROOM_SPEC_SAMPLER` et est appelée au début des générations. Nous avons donc modifié cette variable en supprimant un type de pièce, la cuisine, et en ne laissant que les chambres comme possibilité de pièce disponible. Il n'y a non plus aucun changement à la fin sur les générations. Ce qui est visible en annexes sur la figure 20.

Conclusion

ProcTHOR [1] est un générateur de maisons qui crée des scènes physiquement possibles, cohérentes et avec de bons matériaux. Cependant, ses possibilités sont limitées, avec des variables peu fiables et un générateur peu adaptable à notre situation. En effet, il s'agit d'un générateur d'intérieurs domestiques efficace, mais peu adaptable à d'autres types de scènes. Nous avons donc décidé de créer un nouveau générateur, sans reprendre la base de code de ProcTHOR.



FIGURE 5 – Plusieurs générations de maisons de même dimension, mais de répartition aléatoire.

5 Réalisation d'un générateur

ProcTHOR [1] donnant de moins bons résultats que ce que nous espérons, nous avons décidé d'intégralement créer un générateur de scènes d'intérieur afin de pouvoir comparer ses résultats à la littérature actuelle, bien plus axée sur les scènes d'intérieur de maison et de bureaux. Ce générateur doit reprendre les critères décrits à la partie 3.1, tout en évitant les défauts de ProcTHOR [1]. Il doit donc être très général et facilement modulable. Le travail effectué s'est fait en plusieurs grandes étapes qui sont d'abord la création d'une pièce avec la possibilité de l'exporter au format objet (`.obj`), pour ensuite le transformer en nuage de points. Puis l'importation et le placement de meubles au sol et enfin l'ajout d'objets sur les meubles. La structure du code généré est visible sur la figure 9 et les différentes étapes de réalisation du projet sont décrites à la figure 10. Nous allons détailler plus en détail ces étapes dans les parties ci-dessous.



FIGURE 6 – Génération de maisons en modifiant la variable globale `AVERAGE_ROOM_SIZE`. La valeur de la variable selon les générations est de 3 pour la (A), 5 pour la (B), 10 pour la (C) et 9 pour la (D).

5.1 Création et exportation de la scène

La base du projet était de pouvoir créer une scène 3D qui puisse être intégralement exportée en tant qu'objet afin de lui appliquer la simulation du laser pour en faire un nuage de points.

Création de la scène

La scène est actuellement une pièce très basique qui est rectangulaire, de longueur et de largeur aléatoire entre deux variables définies en amont qui sont les variables `width_limits` et `length_limits`. La hauteur des murs est elle définie par la variable `height_limits`. Ensuite, des matériaux aléatoires sont choisis pour le sol et les murs, sélectionnées dans des dossiers comprenant des matériaux sur Unity. Le résultat des générations est visible sur la figure 11 où il est visible que les pièces ont différentes proportions.

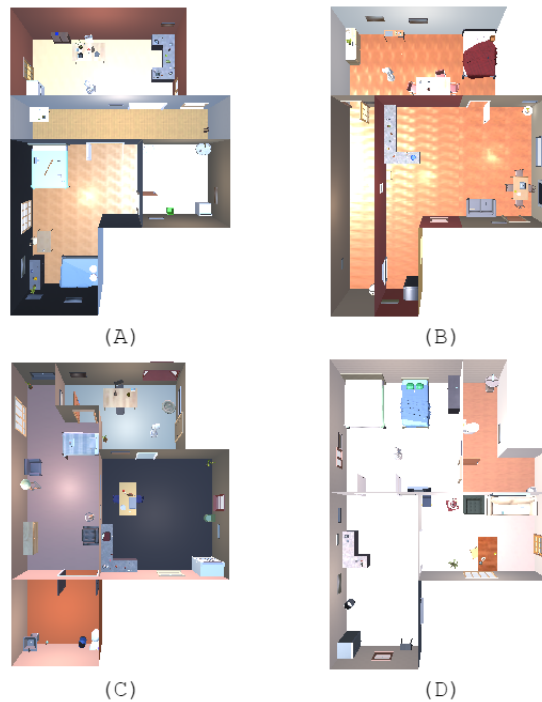


FIGURE 7 – Générations avec la seed 42.

Exportation de la scène

Le package Scene OBJ Exporter sur Unity permet d'exporter l'intégralité de la scène existante pendant l'exécution au moment voulu. Cependant, il reste pour l'instant manuel, l'exportation n'ayant pas été automatisée. Le package donne des résultats corrects lors de l'exportation, cependant il n'est pas à jour pour les textures. Seuls les matériaux basiques à base de couleurs peuvent être exportés. Le résultat de trois exportations ouvertes sous Blender est visible sur la figure 12. Ces exportations permettent d'utiliser le laser virtuel dessus pour en faire un nuage de points, ce qui est ce dont nous avons besoin pour le projet. La figure 13 montre un exemple d'une scène en 3D qui a été transformée en nuage de points. Tous les meubles sont visibles, et la position des capteurs virtuels aussi, à la position des cercles avec moins de points sur le sol.

5.2 Importation et placement des meubles

Un des points les plus importants est de pouvoir importer les meubles et objets du jeu de données de son choix pour pouvoir les placer. Leur placement doit être physiquement possible et cohérent par rapport à des scènes du

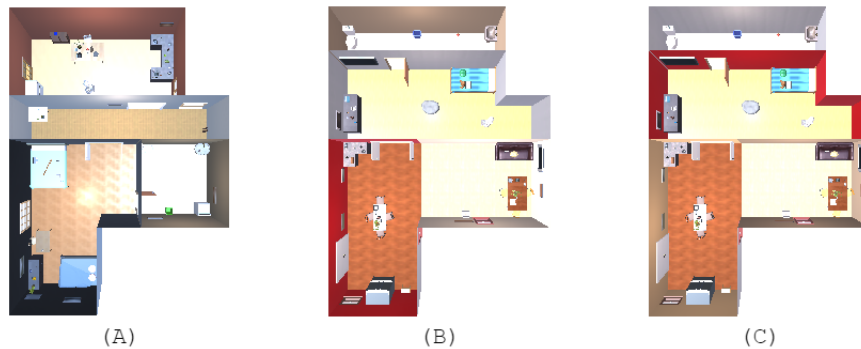


FIGURE 8 – Génération de maisons en modifiant la variable `P_ALLOW_HOUSE_PLANT_GROUP`. Les valeurs de la variable sont de 0.5 pour l'image (A), 10 pour l'image (B), et 0.1 pour l'image (C). La génération (A) possède une plante et pour les générations (B) et (C) deux plantes.

monde réel pour un meilleur apprentissage. Pour cela, il est nécessaire que les objets qui proviennent de bases de données soient des `.obj`. Chaque type d'objet doit être rangé dans un dossier au nom du type de l'objet afin que le générateur puisse les importer sur Unity.

Importation

Afin d'importer des objets, nous utilisons le package `ObjLoader` qui permet d'importer des objets dans la scène Unity. Les objets doivent être classés par catégorie dans des dossiers qui portent le nom de ces catégories. Les objets importés n'ont pas de collider. Une partie de la base de données de ShapeNet est utilisée. Celle-ci comporte des étagères, des armoires, des chaises, des bureaux, des écouteurs, des ordinateurs portables, des écrans, des tasses, des imprimantes, des canapés, des tables et une clé USB. Afin de gérer les collisions, nous avons ajouté un `BoxCollider` qui est une boîte qui entoure l'objet selon ses dimensions. Les dimensions de l'objet sont calculées à partir de ses vertices. Le `BoxCollider` est visible sur la figure 14, plus précisément l'image (a) qui représente un `BoxCollider` simple (boîte aux contours verts), ainsi que sur la figure 15. Le `BoxCollider` permet de savoir si deux objets se percutent au moment où ils sont placés pour ainsi les déplacer à nouveau. Leur position à ce stade est définie aléatoirement dans l'espace de la pièce. Initialement, nous souhaitions appliquer un `MeshCollider` qui prend la forme du mesh de l'objet, cependant, Unity ne permet pas d'utiliser le `MeshCollider` sur des objets à plus de 250 faces, ce qui est systématiquement le cas de nos objets importés. Le `BoxCollider` est problématique dans plusieurs cas,

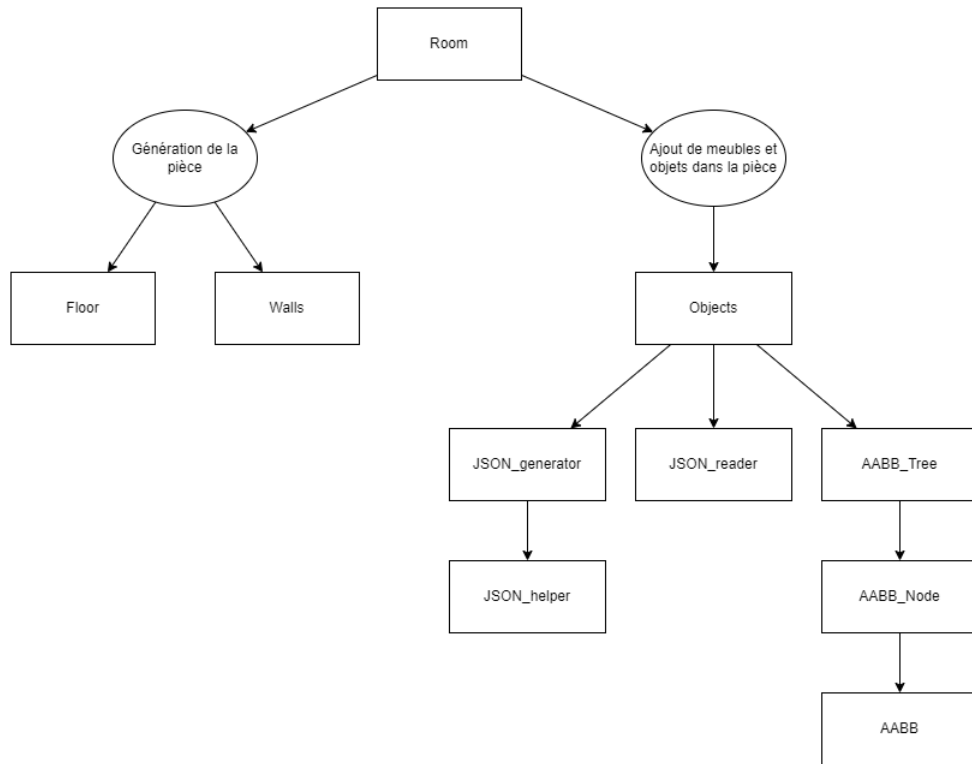


FIGURE 9 – Structure du code du générateur de scènes.

comme pour les canapés en angle qui se retrouve dans une boîte alors qu'ils prennent moins de la moitié de la place, ou pour les objets sur pieds comme les tables qui pourraient avoir de plus petits objets sous eux.

Système de gestion de propriété des objets

Lors de la génération de scènes, un fichier *JSON* est automatiquement créé s'il n'y en a aucun de fourni par l'utilisateur. Ce fichier sert à gérer les propriétés des fichiers. *JSON* reprend les données de chaque objet sous la forme présentée dans le cadre 5.2 ci-dessous :

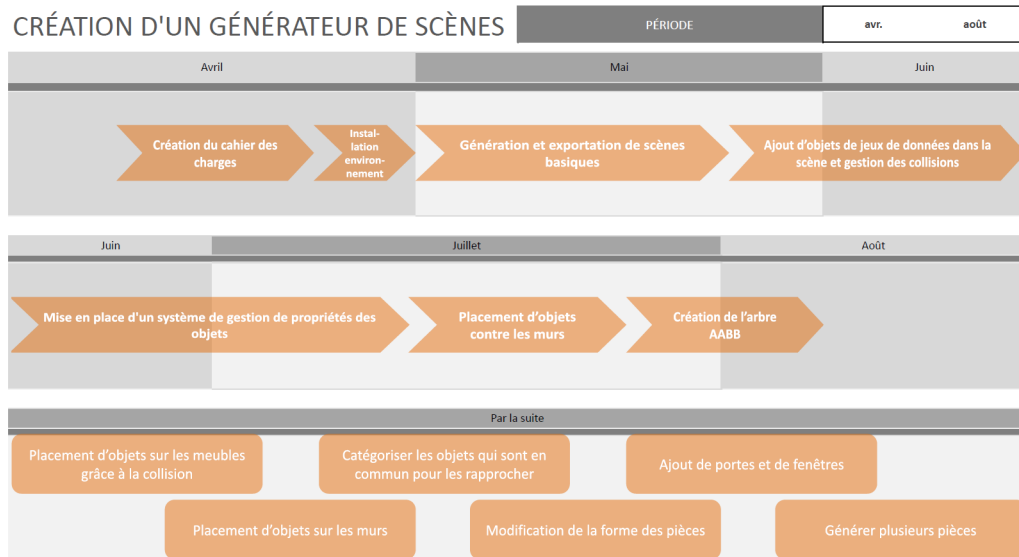


FIGURE 10 – Roadmap du projet et des tâches à venir.

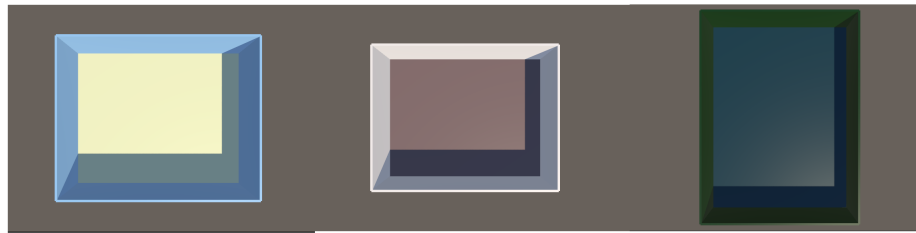


FIGURE 11 – Trois générations de pièces vides.

```
public class ObjectData {
    public string name;
    public string type;
    public bool get_objects;
    public bool on_floor;
    public bool on_objects;
    public int on_ceiling;
    public int on_walls;
    public int against_wall;
    public float size;
    public string tag;
}
```

Pour chaque objet ajouté dans la scène, il y a une référence dans le *JSON* qui

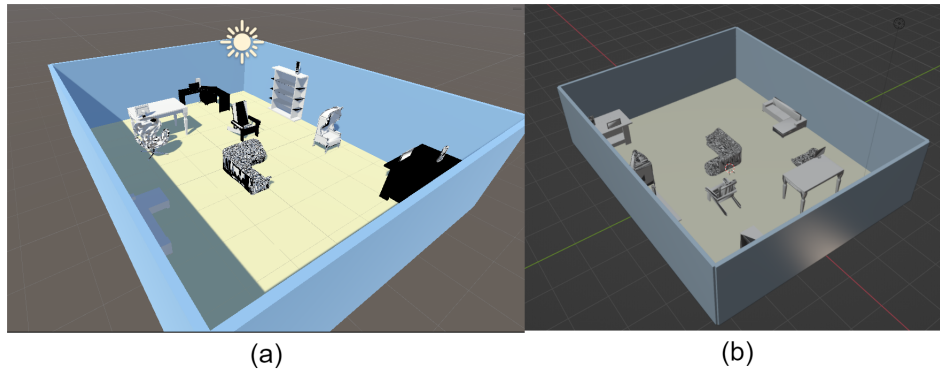


FIGURE 12 – En (a) il y a la scène générée sur Unity, en (b) il y a la même scène exportée et ouverte sous Blender.

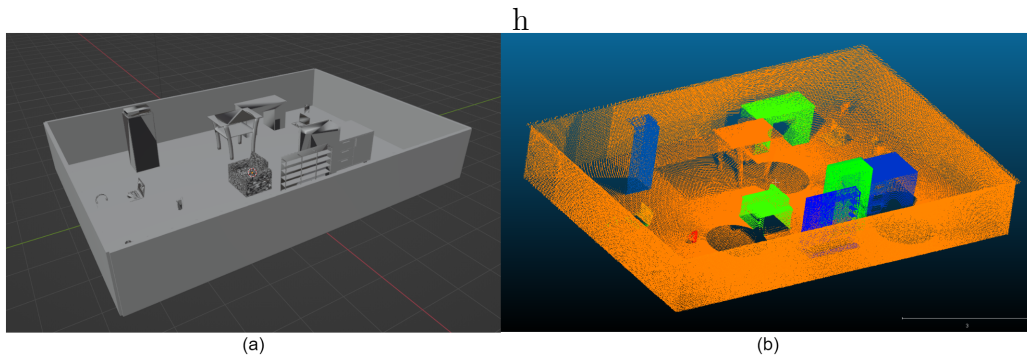


FIGURE 13 – Transformation d’une scène 3D en nuage de points. À gauche en (a), la scène 3D utilisée pour mettre les capteurs ouverte sous Blender, à droite en (b), le nuage de points généré ouvert sous CloudCompare.

décrit son nom, son type, s’il peut recevoir des objets ou non, s’il peut être placé au sol, sur des objets, au plafond, accroché à un mur ou posé contre le mur. Il y a aussi sa taille et la catégorie à laquelle il appartient (le tag). Ces variables regroupent donc les données qui permettent de savoir où peut être placé un objet et ses dimensions. Les données sont ensuite reprises et lues par le générateur afin de placer et redimensionner les objets. Pour lire les données *JSON*, nous avons créé une classe *JSON_helper* à partir de ce code. Cette classe surcharge la fonction *Json_reader* de Unity.

5.3 Implémentation des objets sur les meubles

Afin de déposer des objets sur les meubles, Les différents objets du jeu de données ont été divisés en deux catégories. Celle des meubles qui sont posés

au sol, et celle des objets qui se posent sur des meubles. Cette information est visible dans la variable `tag` sur le *JSON*. Il y est aussi indiqué si un objet peut en recevoir un autre sur lui. Chaque objet choisi donc aléatoirement un meuble sur lequel se placer, et est déposé aléatoirement sur la surface du **BoxCollider** du meuble. Les objets sont placés selon la hauteur maximale du meuble, soit la hauteur du **BoxCollider**. Cela pose régulièrement un problème étant donné que les objets se retrouvent fréquemment à flotter au-dessus des meubles. Par exemple, si un objet se pose sur une chaise, il sera à la hauteur du dossier. Un exemple est visible sur la figure 14.

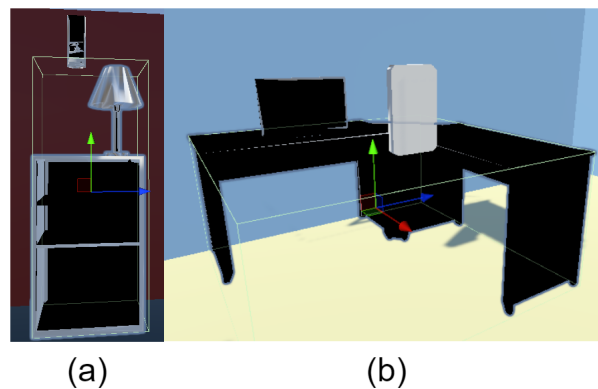


FIGURE 14 – Exemple d’objets volants dus au **BoxCollider**.

5.4 Arbre AABB pour créer les **BoxColliders**

Afin d’affiner le collider et de lui permettre de mieux s’adapter à la forme de l’objet, nous avons dans un premier temps pensé à diviser le **BoxCollider** en huit, vérifier s’il y a des vertices dedans, puis réitérer trois à quatre fois sur les **BoxColliders** restants. Cependant, les vertices ne sont pas présents partout sur l’objet, mais seulement aux angles. Il n’y a donc pas de **BoxCollider** au centre des surfaces planes telles que le plateau des tables. On peut voir la position des vertices sur la figure 15 où chaque sphère blanche représente un vertex de l’objet. Il fallait donc trouver une autre solution.

Poser un **BoxCollider** sur chaque triangle existant de l’objet est trop gourmand et n’est donc pas faisable. Cependant, pour des travaux futurs, il serait intéressant de faire des bounding box orientés pour améliorer l’efficacité et la précision des **BoxColliders** créés. S. Gottschalk et coll. [15] ont travaillé sur un algorithme permettant de calculer une représentation hiérarchique des modèles à l’aide d’arbres de délimitation orientés (OBBTrees).

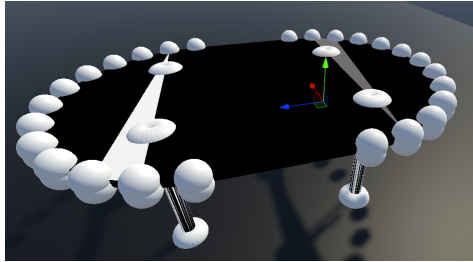


FIGURE 15 – Affichage des vertices. Chaque sphère blanche représente un vertex de l'objet.

AABB_Tree

Nous avons décidé de créer un arbre binaire qui serait généré à partir des triangles de l'objet. Des **BoxColliders** seraient ensuite créés selon un certain niveau de l'arbre pour améliorer la précision tout en limitant les capacités demandées. Nous avons repris ce code en C++ qui a été traduit en C#. La racine de l'arbre correspond à un **BoxCollider** de la taille de l'objet, puis chaque niveau précise les proximités entre les triangles, allant jusqu'à doubler le nombre de **BoxColliders** qui seront de plus petite taille et plus adaptés à l'objet. Il y a des exemples du rendu des **BoxColliders** avec un niveau 0, 10 et 100 sur les objets sur la figure 16. Cependant, plus on descend bas dans les

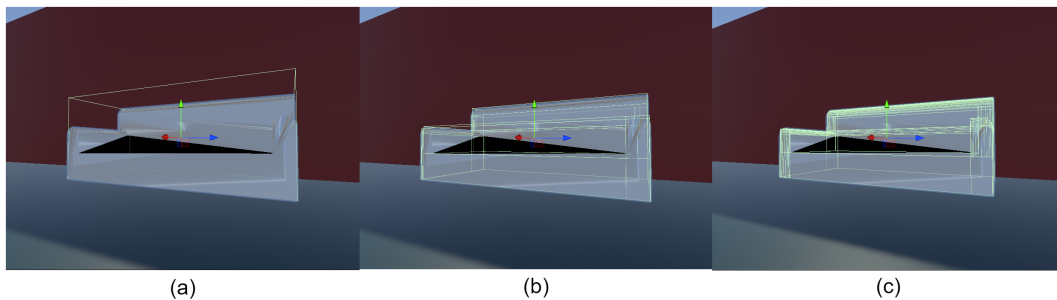


FIGURE 16 – Génération d'un canapé avec une profondeur d'arbre AABB variable pour créer les **BoxColliders**. En (a) il y a 0 de profondeur, en (b) il y a 10 de profondeur et en (c) il y a 100 de profondeur.

niveaux, plus il devient difficile d'exécuter le code qui devient de plus en plus lourd. Le tableau 6 décrit le temps de génération de quatre pièces identiques avec quatre niveaux de profondeur différents. Il démontre bien que très vite la génération devient très longue, la différence est notamment visible entre la profondeur 0 et la profondeur 10 qui passe d'environ une minute à plusieurs minutes, allant jusqu'à 20 minutes pour la première génération. Et avec une

Profondeur	Génération 1	Génération 2	Génération 3	Génération 4
0	1.01 min	4.05 min	0.72 min	0.24 min
4	1.31 min	4.20 min	1.03 min	0.33 min
10	20.6 min	4,15 min	8.03 min	11.00 min
50	> 1h25 (Arrêté)	Non faite	Non faite	Non faite

TABLE 6 – Temps de génération d’une scène comportant 15 objets selon la profondeur de l’arbre AABB.

profondeur de 50, il faut plus d’une heure vingt pour la première génération, ce qui est conséquent.

Il est donc nécessaire de trouver une profondeur optimale qui soit un compromis entre la précision de la forme de l’objet et les capacités de calcul de l’ordinateur. Il pourrait aussi être intéressant d’optimiser davantage le code, potentiellement en faisant un pré-calcul et en sauvegardant les colliders. Il sera nécessaire en amont de vérifier quelle partie du code est la plus longue à exécuter pour pouvoir améliorer cette partie-là.

5.5 Conclusion

Contributions

Actuellement, chaque génération effectuée par le générateur avec la même graine aléatoire donne le même résultat, il y a la possibilité d’importer des objets provenant des jeux de données du choix de l’utilisateur. Les variables sont fiables et simplement accessibles dans le code, ce qui permet de modifier les générations simplement. Cependant, le générateur n’est pas encore terminé, il lui manque beaucoup de points qui seront à ajouter dans le futur. Le résultat des générations actuelles est visible en annexes sur la figure 22 en annexes.

Travaux futurs

Par la suite, il serait intéressant de créer différentes formes de pièce, afin d’éviter de n’obtenir que des pièces rectangulaires. Il serait aussi bien d’ajouter des fenêtres et des portes aux murs et de permettre de poser des objets au mur, comme des étagères ou des horloges. Il faut aussi catégoriser les objets pour permettre à ceux qui vont ensemble de se rapprocher, par exemple les tables et les chaises, mais aussi pour permettre de définir des pièces qui auraient uniquement un certain type d’objets. Enfin, il faudrait pouvoir générer plusieurs pièces côte à côte pour générer un bâtiment ou un étage et non une

simple pièce. Il serait aussi intéressant de revoir l'exportation pour pouvoir récupérer les textures.

6 Résultats

Plusieurs apprentissages ont été réalisés afin de voir l'impact des données générées sur l'apprentissage en Deep Learning pour la compréhension de scènes complexes. Les apprentissages ont été réalisés en suivant le protocole décrit à la partie 3.1.

Dans un premier temps, trois apprentissages test ont été réalisés en utilisant uniquement les données de S3-DIS, puis trois avec une génération de meubles déposés aléatoirement au sol, trois avec une répartition de meubles dont certains se mettent contre les murs, et trois dernières avec des objets se déposant sur des meubles et la possibilité de mettre certains meubles contre le mur. Les résultats sont visibles sur le tableau 7.

/	Données S3-DIS	Avec les meubles aléatoires	Avec les meubles contre le mur	Avec les objets sur les meubles
Accuracy	80,92	79,29	79,05	-
Mean-Accuracy	61.91	61.45	61.51	-
mIoU	21.62	50.33	50.35	-
Tuyau	0.0	0.04	0.01	-
Tableau	57.76	53.86	51.46	-
Bibliothèque	63.24	59.77	60.86	-
Plafond	73.61	70.26	68.30	-
Chaise	74.52	74.66	73.15	-
Désordre	48.54	46.04	46.22	-
Porte	44.28	42.34	42.16	-
Sol	80.65	79.02	77.49	-
Table	72.16	71.40	72.02	-
Mur	74.96	73.24	73.10	-
Colonne	12.73	11.86	13.61	-
Canapé	25.78	30.96	35.22	-
Fenêtre	42.89	40.92	40.39	-

TABLE 7 – Moyennes obtenues par apprentissage selon les données utilisées.

Sans surprises, le générateur n'étant pas très performant, actuellement les scènes générées ne permettent pas d'améliorer la qualité de l'apprentissage.

L'accuracy étant de 80.92 sans le jeu de données et d'environ 79 avec le jeu de données, nous pouvons noter que les pertes ne sont pas non plus énormes et ne sont pas significatives. Les scènes générées n'ont pas encore de cohérence complète, il y a donc de fortes chances que l'amélioration de ce point améliore l'apprentissage.

Il est cependant intéressant de noter que les variations ne sont pas différentes selon si l'objet est dans le jeu de données créé ou non. Par exemple, les fenêtres n'étaient pas dans les scènes générées, et ont un écart plus important, passant de 42.89 lors de l'apprentissage sans le jeu de données à 40.39 et 40.92 avec le jeu de données. Afin de comparer, nous pouvons citer les bibliothèques qui passent de 63.24 à 59.77 et 60.86.

Un autre point intéressant est que l'apprentissage devient tout de même meilleur lorsque l'objet n'était pas dans la base de données de S3-DIS. Par exemple, les canapés ne font pas partie de cette base de données et une nette amélioration d'apprentissage est visible avec le jeu de données généré. Le jeu de données S3-DIS donne un résultat de 25.78 pour les canapés, le jeu de données avec les objets au sol donne un résultat de 30.96, et celui avec des meubles contre les murs (typiquement, les canapés peuvent se déposer contre un mur) donne le résultat de 40.39.

Bilan

Le jeu de données généré n'a actuellement pas de réel impact sur l'apprentissage pour la compréhension de scènes complexes, mais étant donné que le générateur n'est pas terminé, il n'y a rien de surprenant. Une amélioration dans les générations permettra potentiellement d'améliorer ces résultats qui n'ont actuellement rien d'alarmant.

7 Conclusion

L'objectif de ce stage était de réussir à créer un générateur qui soit capable de créer des scènes variées et qui soit facilement modulable, afin de pouvoir générer des scènes industrielles pour en faire des données d'apprentissage en Deep Learning.

Le stage s'est déroulé en deux grandes étapes. Tout d'abord, l'analyse de ProcTHOR [1] nous a permis de constater que le générateur fait des générations de maison qui sont physiquement possibles et réalistes, même si les scènes restent peu encombrées et que la gestion des grands espaces est difficile. Cependant, les variables sont peu modulables et peu fiables, ce qui rend la modification du générateur pour l'adapter à des scènes industrielles plus

difficiles. Par conséquent, nous avons décidé de ne pas reprendre le générateur de ProcTHOR [1] pour réaliser notre propre générateur.

La seconde étape fût de créer le générateur, sans reprendre les bases de ProcTHOR [1]. Il a été réalisé sur Unity et actuellement, il permet de générer une scène rectangulaire et d'y déposer des meubles dont une partie peut se mettre contre les murs ainsi que de déposer des objets dessus. Un arbre AABB est créé afin de générer des bounding box proches de la forme de l'objet, sans qu'elles soient trop gourmandes. Plusieurs difficultés ont été rencontrées durant ce stage, notamment dû à des limitations de Unity, comme le fait qu'il ne puisse pas y avoir plus de 255 faces par objet pour utiliser le `MeshCollider`.

Ce générateur donne actuellement des résultats qui sont peu intéressants, diminuant même les résultats de l'apprentissage par rapport au jeu de données S3-DIS seul. Cependant, les résultats sont cohérents étant donné que le générateur n'est pas terminé, ne produisant pas des scènes totalement réalistes.

Les améliorations futures proposées sont visibles sur la roadmap 10 dans la partie *Par la suite*. En effet, il serait intéressant de poursuivre le générateur en variant la forme des pièces et en ajoutant des fenêtres, des portes et des objets au mur. Il faudra aussi catégoriser les objets pour permettre de rapprocher les objets selon la pièce qui leur correspond, ou les objets qui sont régulièrement proches dans un environnement classique. Par la suite, générer un étage ou un bâtiment à plusieurs pièces et non uniquement une seule pièce serait intéressant. Améliorer l'exportation des scènes pour récupérer les textures est un autre point important à réaliser. Pour terminer, il serait bien de retravailler la méthode de création des bounding box pour l'alléger et le rendre plus performant, par exemple en reprenant l'arbre AABB, mais en créant des boxs orientées.

Références

- [1] Matt DEITKE et al. “ProcTHOR : Large-Scale Embodied AI Using Procedural Generation”. In : *NeurIPS 2022 Conference Paper3089 Area Chair AgNm*. arXiv, juin 2022. DOI : 10.32130/idr.6.0. arXiv : 2206.06994 [cs].
- [2] John McCORMAC et al. “SceneNet RGB-D : Can 5M Synthetic Images Beat Generic ImageNet Pre-Training on Indoor Segmentation ?” In : *2017 IEEE International Conference on Computer Vision (ICCV)*. 2017, p. 2678-2687. DOI : 10.1109/ICCV.2017.292.
- [3] Zhengqin LI et al. “OpenRooms : An Open Framework for Photo-realistic Indoor Scene Datasets”. In : *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Nashville, TN, USA : IEEE, juin 2021, p. 7186-7195. ISBN : 978-1-66544-509-2. DOI : 10.1109/CVPR46437.2021.00711.
- [4] Angela DAI et al. “ScanNet : Richly-Annotated 3D Reconstructions of Indoor Scenes”. In : *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Honolulu, HI : IEEE, juill. 2017, p. 2432-2443. ISBN : 978-1-5386-0457-1. DOI : 10.1109/CVPR.2017.261.
- [5] Armen AVETISYAN et al. “Scan2CAD : Learning CAD Model Alignment in RGB-D Scans”. In : *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Long Beach, CA, USA : IEEE, juin 2019, p. 2609-2618. ISBN : 978-1-72813-293-8. DOI : 10.1109/CVPR.2019.00272.
- [6] Binh-Son HUA et al. “SceneNN : A Scene Meshes Dataset with aN-Notations”. In : *2016 Fourth International Conference on 3D Vision (3DV)*. Stanford, CA, USA : IEEE, oct. 2016, p. 92-101. ISBN : 978-1-5090-5407-7. DOI : 10.1109/3DV.2016.18.
- [7] Gilad BARUCH et al. “ARKitScenes : A Diverse Real-World Dataset For 3D Indoor Scene Understanding Using Mobile RGB-D Data”. In : *NeurIPS 2021 Datasets and Benchmarks Track*. arXiv, jan. 2022. DOI : 10.48550/arXiv.2111.08897. arXiv : 2111.08897 [cs].
- [8] Bichen WU et al. “SqueezeSeg : Convolutional Neural Nets with Recurrent CRF for Real-Time Road-Object Segmentation from 3D LiDAR Point Cloud”. In : *2018 IEEE International Conference on Robotics and Automation (ICRA)*. arXiv, oct. 2017. DOI : 10.1109/ICRA.2018.8462926. arXiv : 1710.07368 [cs].

- [9] Angel X. CHANG et al. *ShapeNet : An Information-Rich 3D Model Repository*. Déc. 2015. DOI : 10.48550/arXiv.1512.03012. arXiv : 1512.03012 [cs].
- [10] Ankur HANDA et al. “Understanding RealWorld Indoor Scenes with Synthetic Data”. In : *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA : IEEE, fév. 2018, p. 4077-4085. ISBN : 978-1-4673-8851-1. DOI : 10.1109/CVPR.2016.442.
- [11] Kaichun MO et al. *The AdobeIndoorNav Dataset : Towards Deep Reinforcement Learning Based Real-world Indoor Robot Visual Navigation*. Stanford, CA, USA, fév. 2018. DOI : 10.48550/arXiv.1802.08824. arXiv : 1802.08824 [cs].
- [12] Miho ADACHI et al. “Accuracy Improvement of Semantic Segmentation Trained with Data Generated from a 3D Model by Histogram Matching Using Suitable References”. In : *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. Oct. 2022, p. 1180-1185. DOI : 10.1109/SMC53654.2022.9945583.
- [13] Ekta U. SAMANI et Ashis G. BANERJEE. “Topologically Persistent Features-based Object Recognition in Cluttered Indoor Environments”. In : *2021 IEEE Robotics and Automation Letters*. arXiv, mai 2022. DOI : 10.48550/arXiv.2205.07479. arXiv : 2205.07479 [cs].
- [14] Hugues THOMAS et al. “KPConv : Flexible and Deformable Convolution for Point Clouds”. In : *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, p. 6411-6420.
- [15] S. GOTTSCHALK, M. C. LIN et D. MANOCHA. “OBBTree : A Hierarchical Structure for Rapid Interference Detection”. In : *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '96. New York, NY, USA : Association for Computing Machinery, août 1996, p. 171-180. ISBN : 978-0-89791-746-9. DOI : 10.1145/237170.237244. (Visité le 03/08/2023).
- [16] Ryosuke YAMADA et al. “Point Cloud Pre-training with Natural 3D Structures”. In : *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA : IEEE, juin 2022, p. 21251-21261. ISBN : 978-1-66546-946-3. DOI : 10.1109/CVPR52688.2022.02060.
- [17] Lukas WINIWARTER et al. “Virtual Laser Scanning with HELIOS++ : A Novel Take on Ray Tracing-Based Simulation of Topographic Full-Waveform 3D Laser Scanning”. In : *Remote Sensing of Environment*.

- Remote Sensing of Environment 269 (fév. 2022), p. 112772. ISSN : 0034-4257. DOI : 10.1016/j.rse.2021.112772.
- [18] Florian NOICHL, Alexander BRAUN et André BORRMANN. “BIM-to-Scan for Scan-to-BIM : Generating Realistic Synthetic Ground Truth Point Clouds Based on Industrial 3D Models”. In : *EC3 Conference 2021*. T. 2. Computing in Construction. ETH, 2021, p. 164-172. ISBN : 978-3-907234-54-9. DOI : 10.35490/EC3.2021.166.
- [19] Anton KVESIĆ, Danijel RADOŠEVIĆ et Tihomir OREHOVAČKI. “Using SCT Generator and Unity in Automatic Generation of 3D Scenes and Applications”. In : *25th Central European Conference on Information and Intelligent Systems*. Rochester, NY, sept. 2014.

8 Annexes

8.1 Installation de l'environnement de travail pour ProcTHOR [1]

Dans un premier temps, une analyse qualitative de ProcTHOR a été effectuée pour voir s'il serait efficace de reprendre ProcTHOR [1] et de l'adapter pour réaliser un générateur de scènes industrielles. Pour faire fonctionner ProcTHOR il y a besoin de Cuda (**C**ompute **U**nified **D**evice **A**rchitecture) qui permet d'utiliser le processeur graphique pour exécuter des calculs généraux à la place du processeur et d'Anaconda qui permet de simplifier la gestion des paquets et de déploiement.

J'ai donc installé WSL 2 (**W**indows **S**ubsystem **L**inux) qui est un coeur Linux natif sous Windows, et qui permet de faire fonctionner les exécutable binaires de Linux. Ensuite, la distribution pour Linux a été installée, précisément la version d'Ubuntu 20.04, Cuda ne pouvant s'exécuter sur des versions plus récentes, via les lignes suivantes :

```
wsl -l -o
wsl --install -d Ubuntu-20.04
sudo apt update
sudo apt upgrade -y
```

Puis j'ai installé Cuda 11.3 comme ceci :

```
sudo apt-key del 7fa2af80
wget https://developer.download.nvidia.com/compute/cuda
/repos/wsl-ubuntu/x86_64/cuda-wsl-ubuntu.pin
sudo mv cuda-wsl-ubuntu.pin /etc/apt/preferences.d/cuda
-repository-pin-600
wget https://developer.download.nvidia.com/compute/cuda
/11.3.1/local_installers/cuda-repo-wsl-ubuntu-11-3-
local_11.3.1-1_amd64.deb
sudo dpkg -i cuda-repo-wsl-ubuntu-11-3-local_11.3.1-1
_amd64.deb
sudo apt-key add /var/cuda-repo-wsl-ubuntu-11-3-local/7
fa2af80.pub
sudo apt-get update
sudo apt-get -y install cuda
```

La version 11.3 de Cuda a été choisie car il y a besoin de PyTorch, une bibliothèque logicielle Python open source qui permet d'effectuer les calculs tensoriels nécessaires pour l'apprentissage profond. Or, il faut que les versions

de Cuda et PyTorch soient compatibles entre elles et avec l'ordinateur qui les exécutent, ici il s'agit de la version 11.3 de Cuda avec la version 1.12.1 de PyTorch.

Avant d'installer PyTorch, il faut un environnement Anaconda qui se crée en exécutant les lignes de code ci-dessous après avoir téléchargé le fichier nécessaire sur ce site.

```
bash [nom_du_fichier].deb
conda create -p /mnt/d/conda/[env] python=3.X
conda activate /mnt/d/conda/[env]
```

Puis, PyTorch 1.12.1 est installé sur l'environnement via :

```
conda install pytorch==1.12.1 torchvision==0.13.1
torchaudio==0.12.1 cudatoolkit=11.3 -c pytorch
```

Enfin, j'installe ProcTHOR et les packages nécessaires à son bon fonctionnement via un fichier *requirements.txt*.

```
pip install -r requirements.txt
```

Fichier *requirements.txt* :

```
procthor
ai2thor
attr
attrs
pandas
shapely
moviepy
networkx
python-fcl
```

8.2 Structure de ProcTHOR [1]

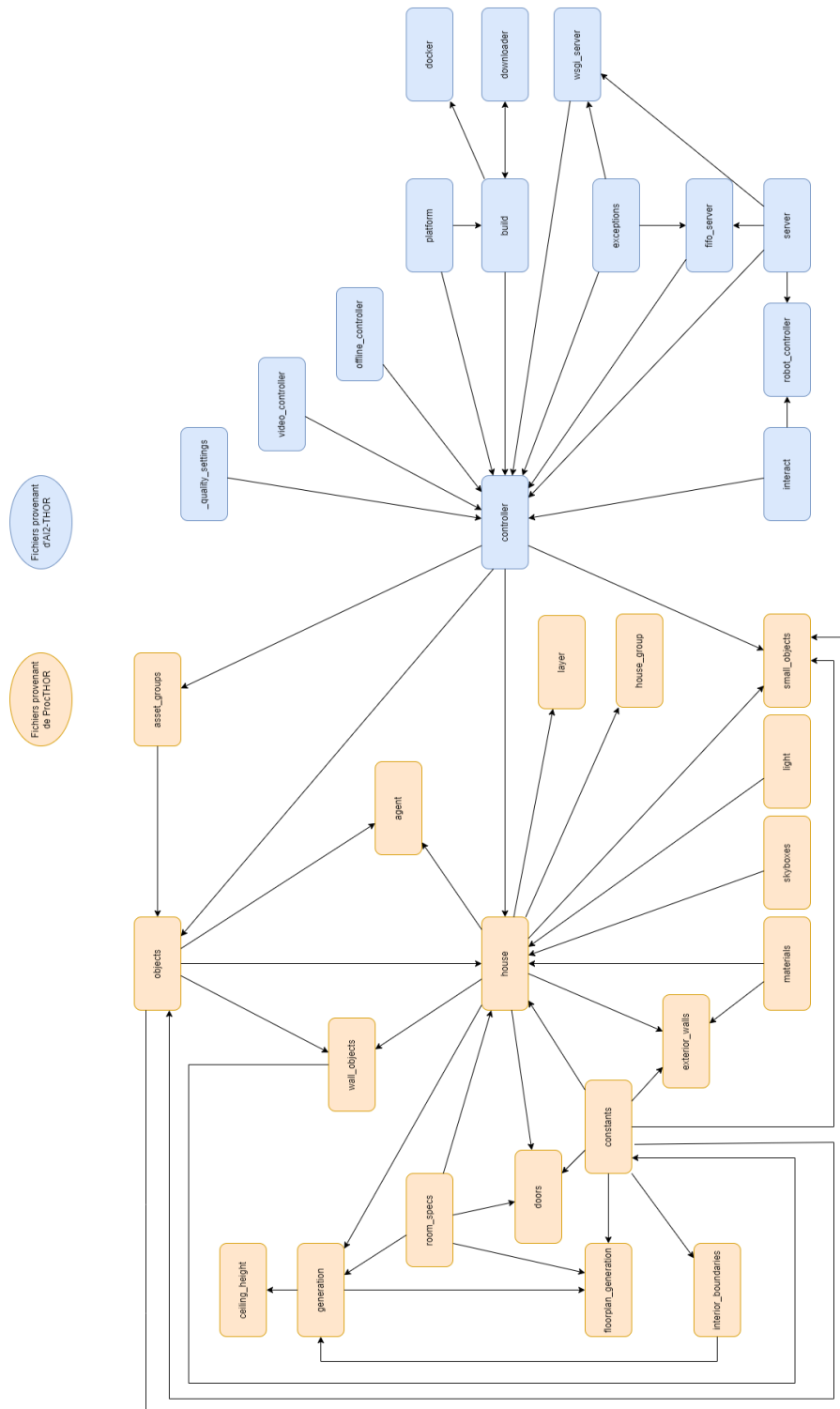


FIGURE 17 – Schéma de la structure du code de ProcTHOR provenant du GitHub.

8.3 Modification des variables de ProcTHOR [1]



FIGURE 18 – Génération de maisons en modifiant la variable `num_rooms`. La valeur de la variable est indiquée sous chaque image de génération prise.

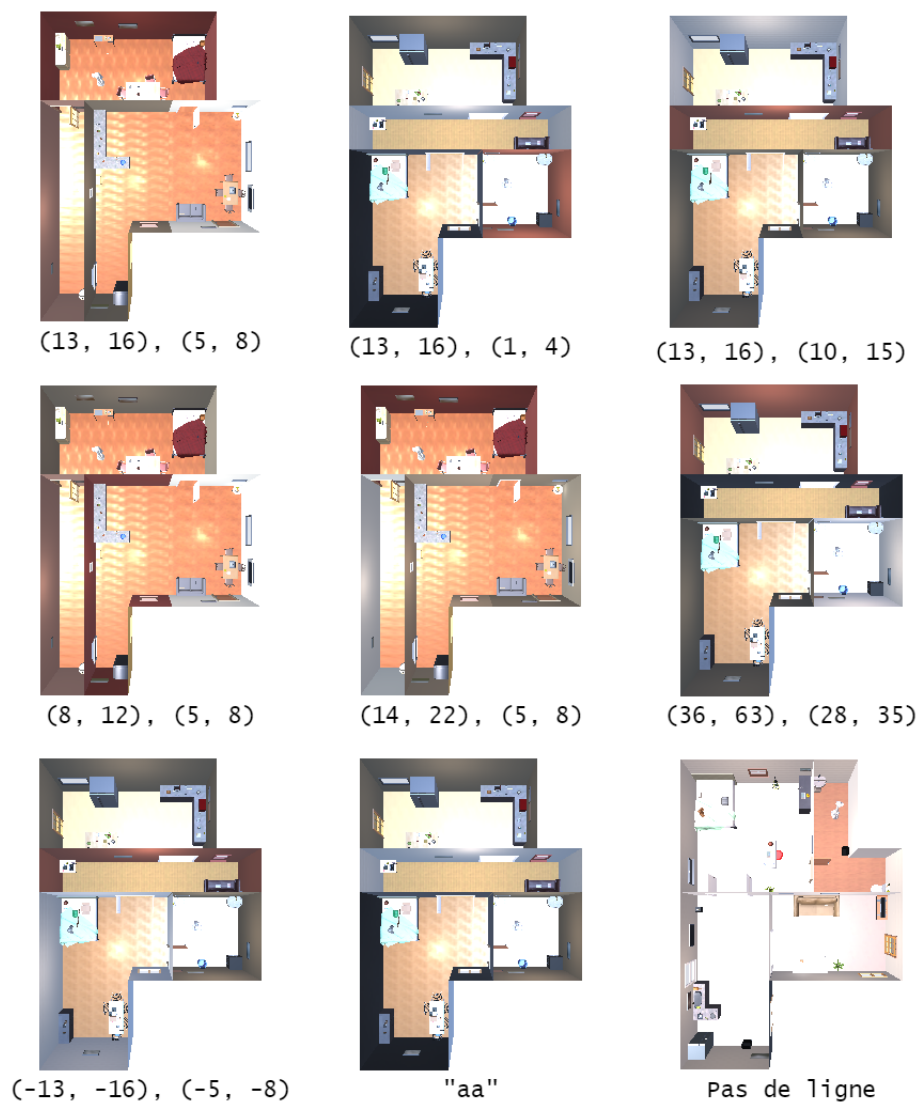


FIGURE 19 – Génération de maisons en faisant varier dims du fichier `room_specs.py`. On peut voir sous les images la valeur de la variable associée à la génération, sachant que lorsqu'il y a des valeurs sous la forme (w, x) , (y, z) , cela signifie en réalité que la variable était égale à `lambda: (random.randint(w, x), random.randint(y, z))`.



FIGURE 20 – Génération de maisons avec les cuisines en moins et avec uniquement des chambres.



FIGURE 21 – Génération avec une seule texture de mur existante en utilisant les seeds 42 et 22.

8.4 Générations de scènes avec des objets sur les meubles

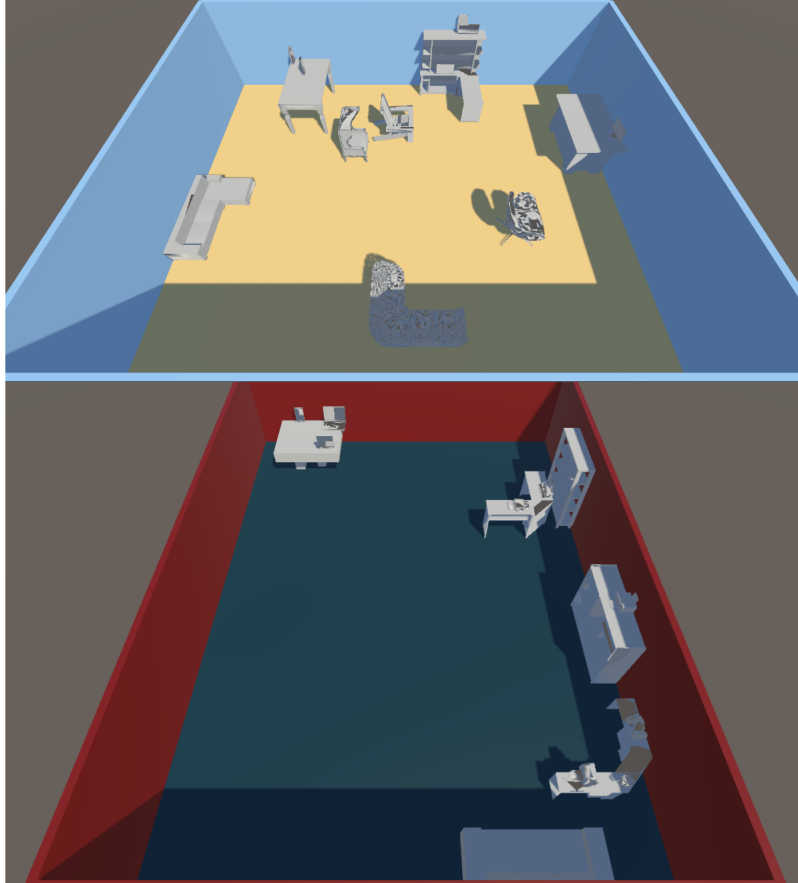


FIGURE 22 – Générations les plus poussées actuellement, avec des meubles qui se mettent contre les murs et des objets qui se posent sur les meubles.

8.5 Bibliographie

8.5.1 Abstract

Depuis quelques années, la recherche en compréhension de scènes complexes par des réseaux de neurones s'est grandement développée. Afin de permettre au réseau d'apprendre, il faut lui mettre à disposition des jeux de données qui soient complets, qui correspondent aux besoins de l'utilisateur et soient correctement annotés. De plus en plus de jeux se sont développés ces dernières années, et la recherche se poursuit dans ce domaine. Ce papier est une étude bibliographique qui résume les différents jeux de données servant

à la compréhension de scènes d'intérieur, et essaie de ressortir les points importants pour évaluer l'efficacité des jeux proposés, tels que les annotations, qui sont primordiales et qui doivent être détaillées. Il y a aussi méthodes de capture des données (données du monde réel, données générées, récupération d'autres jeux de données) et le type des données à disposition (images, vidéos, cartes de profondeur, nuages de points, données thermiques). Enfin, il y a la qualité des données fournies (les angles de vue, le travail de la lumière, des matériaux et le bruit).

8.5.2 Introduction

Depuis quelques années, les chercheurs ont commencé à s'intéresser à la compréhension de scènes, qu'elles soient d'intérieur ou d'extérieur. Les scènes complexes sont des scènes qui sont composées de nombreux objets, et qui appartiennent au quotidien. Ce peut être des scènes d'intérieurs domestiques, par exemple une cuisine avec tous les appareils électroménagers, les tables, le four. Sinon des scènes d'extérieur, notamment urbaines, comme la rue, avec la route, les trottoirs, les passants, les voitures, les animaux. Enfin, il peut exister de nombreuses autres scènes, comme des scènes industrielles principalement composées de tuyaux, de machines en tout genre tels que des vannes, des équipements électriques, des structures métalliques, des convoyeurs ou des pompes. Ce sont des scènes qui peuvent facilement être encombrées, avec un risque élevé d'obstruction sur de nombreux éléments. Le défi actuel est donc de pouvoir identifier chaque élément à partir de ce dont on peut voir d'eux, et de pouvoir se repérer dans l'espace.

La compréhension de scènes d'intérieur a commencé avec Kinect en 2008. L'appréhension de l'environnement dans lequel évolue l'utilisateur, par exemple en réalité augmentée, est devenue un point central depuis quelques années. Il en va de même en robotique, qu'il s'agisse de robots ménagés ou de voitures autonomes par exemple. L'utilisation du Deep Learning, ou apprentissage profond, permet d'arriver à de bons résultats pour la compréhension de scènes complexes. Mais pour que cela fonctionne, il faut avoir des jeux de données qui soient correctement annotés pour l'apprentissage, et en bonne quantité. Aujourd'hui, peu de jeux de données sont vraiment performants, et les recherches se poursuivent pour créer des jeux de données permettant d'apprendre à comprendre un large spectre de scènes, qu'elles soient encombrées ou non.

Il existe différents types de données qui peuvent être récupérées sur une scène, et qui varient selon les jeux de données. Des données RGB (Red, Blue Green), qui sont de simple captures photo et vidéo d'une scène [8, 1]. Des données RGB-D (Red Blue Green Depth), qui sont des données RGB avec en

plus la capture de la profondeur des scènes. Ce sont des images ou des vidéos dont des données de profondeur sont indiquées sur une carte de profondeur de chaque image [2, 3, 4, 6, 12, 7, 10, 13, 9]. Des données uniquement basées sur la profondeur [11, 16, 17, 18]. Des données thermiques présentées sur une carte thermique de la scène comme dans le jeu de données de Armen Avetisyan et coll. dans l'article présentant Scan2CAD [5]. Les données présentes dans les jeux de données peuvent être des données créées de toute pièce [2, 19, 1, 17]. Certains jeux de données sont des captures du monde réel qui ont été modélisés en 3D [3, 5, 10]. D'autres conservent les données récupérées, et les amplifient [3, 5, 9, 1, 18].

La compréhension des scènes complexes avec du Deep Learning doit passer par l'apprentissage avec un jeu de données, et depuis quelques années, de nombreux jeux variés voient le jour. Il existe de nombreux critères possibles pour la sélection du bon jeu de données, pouvant aussi dépendre de l'utilisation qui en est faite. Dans cet article, nous allons tenter de passer en revue les différents critères qui définissent une scène, et nous allons comparer les différentes propositions de plusieurs jeux de données, ainsi que leurs résultats.

Cette étude bibliographique se fait dans le cadre d'un stage sur la compréhension de scènes complexes industrielles par un réseau de neurones. Suite à une recherche sur la conversion de scènes mesh en nuages de point, nous allons nous intéresser sur plusieurs jeux de données avec des types variés. L'organisation de cette étude bibliographique est la suivante : Dans un premier temps, nous verrons plusieurs méthodes de capture de données, ensuite, nous verrons les méthodes de créations pour l'annotation des scènes, puis la qualité des données fournies dans les différents jeux de données, et nous terminerons avec un bilan sur les différents axes observés.

8.5.3 Capture de données

L'acquisition est une étape importante pour la constitution d'un jeu de données. En effet, ils doivent être constitués de nombreuses données, car plus il y en a, mieux c'est pour l'apprentissage, et elles doivent être de bonne qualité. Il existe donc plusieurs possibilités pour en capturer soi-même, en générer par ordinateur, ou en récupérer dans des jeux de données déjà existants. Cependant, ces différentes méthodes de capture ont un coût différent en temps de création, ainsi qu'en coût humain.

Données du monde réel

Afin de capturer des données du monde réel, il existe plusieurs appareils qui peuvent être utilisés, comme des caméras classiques, qui permettent de capturer les données RGB, soit les images, que ce soit en photo ou en vidéo. Ce sont les appareils qui ont été utilisés pour la création de ArkitScenes par Gilad Baruch et coll. [7], par Miho Adachi et coll. [12] et Ekta U. Samani et coll. [13]. Les images et vidéos capturées doivent être assez nettes, et les directions que prennent les caméras doivent être un minimum variées. Cependant, elles sont souvent limitées par des contraintes humaines, étant donné qu'il y a un fort coût humain pour aller récupérer les données nécessitant des déplacements vers des lieux variés ou des créations de plusieurs environnements, ainsi que du temps pour prendre les captures avec l'appareil à configurer, et parfois, il faut faire appel à beaucoup de personnes pour avoir une certaine quantité de données.

Il y a aussi les caméras RGB-D qui, de la même manière que pour les données RGB, permettent de capturer la carte de profondeur de la scène en plus de son image. Ces caméras ont été utilisées pour les jeux de données des articles de Kaichun Mo et coll. [11], Angela Dai et coll. pour ScanNet [4] et Armen Avetisyan et coll. pour Scan2CAD [5]. Comme la méthode est similaire, il y a les mêmes contraintes que pour la capture de données RGB.

Il est aussi possible d'utiliser des capteurs de profondeur, comme les capteurs LIDAR (Light Detection and Ranging), qui renvoient un nuage de points 3D de la scène scannée. Cela crée une carte de profondeur en 3D. Ces capteurs sont souvent onéreux, et ont été utilisés par Gilad Baruch et coll. pour ArkitScenes [7], Bichen Wu et coll. pour SqueezeSeg [8], Miho Adachi et coll. [12] et Lukas Winiwarter et coll. pour HELIOS++ [17], un framework qui simule un laser afin de récupérer des données de profondeur. Les données en nuages de points ont tendance à être moins dépendantes de l'environnement, cependant sont plus facilement obstruées, et peuvent avoir des trous dans les données. De plus, il y a les mêmes conditions de récupération que pour les deux techniques citées au-dessus, il y a donc un coût humain important, et il faut beaucoup de temps pour récupérer les données.

Pour la plupart, les données récupérées seront conservées et améliorées, en complétant les données manquantes, notamment les trous dus à des obstructions. Une autre partie, comme Armen Avetisyan et coll. pour Scan2CAD [5], va créer des scènes en 3D à partir des données récupérées, ainsi qu'une carte thermique en lien avec la scène.

La capture de données du monde réel est avantageuse pour le réalisme d'une scène. Cependant, il y a souvent des trous dans les données capturées, notamment à l'endroit du capteur, ou lorsque des objets obstruent d'autres

objets, notamment dans les scènes encombrées. Une partie ne sera plus visible, ce qui peut rendre la compréhension et l'apprentissage plus difficile, mais qui est pourtant courant dans la vie réelle. De plus, les captures faites à la main demandent beaucoup de temps, étant donné qu'un humain doit se déplacer dans différents lieux, ou modifier un même lieu pour pouvoir créer plusieurs données. C'est notamment lié au fait de devoir se déplacer, ou aménager des scènes, configurer les appareils nécessaires pour la capture et la réalisation de ces différentes captures. Nous pouvons citer Angela Dai et coll. dans l'article "ScanNet : Richly-annotated 3D Reconstructions of Indoor Scenes" [4] qui ont dû demander à une vingtaine d'utilisateur de capturer différentes scènes d'intérieur. Il a donc fallu quatre mois pour la récupération de 1513 scans par des personnes volontaires avant de pouvoir passer aux annotations. Cela demande par conséquent plus de temps et plus de moyens humains, devant équiper chaque humain participant de capteurs adéquats, et éventuellement les former pour qu'ils fassent une capture correcte. Les capteurs eux-mêmes ont un coût financier qu'il faut pouvoir assurer. De plus, la quantité de données est limitée avec 39 000 images pour ArkitScene de Gilad Baruch et coll. [7] et 10 000 images pour l'article "Understanding RealWorld Indoor Scenes With Synthetic Data" de Ankur Handa et coll. [10], par rapport à des jeux de données comme Matt Deitke et coll. pour ProcTHOR [1] qui ont plus de 10 000 maisons intégralement modélisées en 3D pour faire des captures.

Données synthétiques

Il est possible de générer des données par ordinateur pour créer un jeu de données. Pour les réaliser, il faut récupérer des données afin que les scènes soient les plus cohérentes possible. Par exemple, pour des scènes d'intérieur, il est bien d'avoir des données et statistiques sur la présence de meubles en fonction de la pièce qui est créée, afin qu'il n'y ait pas un micro-ondes dans la salle de bain. Par exemple, dans l'article de John McCormac et coll. sur SceneNet RGBD [2], chaque meuble appartient à une ou plusieurs catégories dans lesquelles nous allons chercher lorsque nous créons une pièce. Il en va de même pour la quantité de meubles présents dans l'espace au mètre carré. De plus, les meubles ont une direction, ils sont en général posés dans le même sens, par exemple, une table aura toujours ses pieds au sol. Il faut donc faire en sorte qu'ils soient bien positionnés pour un meilleur apprentissage. Dans SceneNet RGBD [2], le point de gravité des meubles a été abaissé afin qu'ils soient mis dans le bon sens à chaque génération. Cependant, ils ne seront pas pour autant disposés de manière cohérente. Or, la disposition des meubles a aussi son intérêt, notamment pour l'apprentissage. Par exemple, la table

ne sera pas posée sur la télévision, quand bien même c'est physiquement possible. Sur ce point-là, John McCormac et coll. [2] n'obtiennent pas de résultat satisfaisant, misant sur la quantité plutôt que la qualité, contrairement à Matt Deitke et coll. pour ProcTHOR [1] qui génèrent des scènes de biens domestiques cohérentes avec un placement d'objets plausible, naturel et réaliste.

Dans les deux exemples que nous avons vus de génération automatique de scènes, les données qui sont générées afin de servir pour l'apprentissage sont des données RGB-D dans le cas de John McCormac et coll. pour SceneNet RGBD [2], et des données RGB dans le cas de Matt Deitke et coll. pour ProcTHOR [1]. Le nombre de données est plutôt conséquent, avec, par exemple, 70 000 images provenant de six types d'intérieur pour John McCormac et coll. sur SceneNet RGBD [2], ou alors la création de 10 000 maisons possédant entre 1 et 10 pièces ainsi que des dizaines d'objets par pièce pour Matt Deitke et coll. [1], ce qui est bien plus conséquent que les captures réalisées dans le monde réel.

Nous pouvons aussi voir dans l'article "Using SCT Generator and Unity in Automatic Generation of 3D Scenes and Applications" d'Anton Kvesić et coll. [19] une proposition de génération textuelle de scènes en se servant du moteur de jeu multiplateforme Unity pour recréer les scènes en 3D. Le choix de Unity se justifie par le fait qu'il peut être géré de manière programmatique, c'est-à-dire qu'on peut modifier la scène 3D générée directement dans l'éditeur.

Le générateur créé se base sur les générateurs SCT (Specification, Configuration, Templates) qui est un modèle de cadres dynamiques. Il permet de séparer le code du générateur de la configuration, et ainsi de préciser complètement le processus de génération via une syntaxe de configuration simple, au lieu d'une programmation de configuration manuelle. Cette technique permet la génération et l'exécution du code de programmation selon les envies de l'utilisateur, et elle convient aussi à la création d'applications composées de différents types de code comme pour du web.

Puis il faut réussir à faire le lien avec le logiciel de modélisation 3D, tout d'abord en définissant ses capacités, puis connecter ce logiciel et le générateur via un langage commun.

Ensuite, on charge la scène 3D générée dans Unity via l'ajout et la suppression d'artefacts 3D et la modification de caractéristiques comme la rotation, la texture ou la position.

Enfin, un exécutable est créé et le code du programme est créé. Le tout peut être compilé sans passer par Unity.

Il s'agit d'une méthode différente pour la génération de scènes qui est uniquement textuelle, sans capture en amont, et uniquement via du code. Le temps

nécessaire à la création des scènes dépend donc du temps mis par les différents programmeurs pour générer des pièces, sachant que les positions des objets, leurs textures, et toutes les informations dont nécessitent une pièce sont définies par des humains, et non générées automatiquement.

L'article "Point Cloud Pre-training with Natural 3D Structures" de Ryo-suke Yamada et coll. [16] présente un autre jeu de données, nommé PC-FractaleDB, qui renvoie un nuage de points 3D créé à partir de scènes générées en se basant sur les structures naturelles de fractales, une formule mathématique. Un modèle fractal 3D est généré automatiquement, et est expliqué comme ceci : "(1) Plusieurs transformations affines et probabilités de sélection sont définies de manière aléatoire. (2) Le nuage de points initial est indiqué par les coordonnées d'origine et est défini comme le nuage de points actuel. (3) L'une des transformations affines est sélectionnée en fonction des probabilités de sélection. (4) Le nuage de points actuel est transformé de manière affine pour le nuage de points suivant en utilisant la transformation affine sélectionnée. (5) Les étapes 3 et 4 sont effectuées de manière récursive jusqu'à l'ensemble des N itérations. Les étapes 3 et 4 sont effectuées de manière récursive jusqu'à l'ensemble des N itérations." dans l'article de Ryo-suke Yamada et coll. [16]

Les données synthétisées ont l'avantage d'être plus conséquentes, le coût humain étant plus léger, et la génération de scènes plus rapide que la capture. Par exemple, Matt Deitke et coll. pour ProcTHOR [1] ont pu générer les 10 000 habitations en une heure. Nous pouvons donc avoir de plus grands jeux de données. Il s'agit donc d'une solution pour générer rapidement une grande quantité de données, parfois avec des données de bonne qualité, avec des scènes réalistes. De plus, les annotations sont bien plus rapides à réaliser étant donné que les données sont directement disponibles, que ce soit à la création d'une scène, ou à la récupération de données de scènes 3D déjà existantes.

Cependant, la génération de scènes peut être particulièrement gourmande au niveau technologique. De plus, il peut être difficile d'obtenir des scènes réalistes, ce qui peut nuire à l'apprentissage, mais qui n'est pas totalement impossible, comme nous l'ont montré Matt Deitke et coll. avec ProcTHOR [1] qui se sont axés sur la génération de scènes d'intérieur de domiciles.

Données d'un jeu vidéo

Certains jeux vidéos ont une représentation très réaliste de la réalité, et il est possible d'en récupérer les données pour entraîner des intelligences artificielles. C'est sur cette idée que s'est basé [8] pour récupérer plus de données au-delà de celles déjà en leur possession, afin d'apprendre à un véhicule de

circuler de manière autonome sur la route. Bichen Wu et coll. ont capturé des données en simulant un capteur de profondeur dans le jeu GTA V de Rockstar Games, dans lequel une attention particulière a été mise sur les voitures. Ainsi, ils ont pu récupérer rapidement une bonne quantité de données, ce qui a permis d'améliorer grandement la détection des voitures par l'intelligence artificielle. Cependant, les cyclistes et les piétons n'ont pas pu être mieux détectés étant donné qu'ils sont très peu détaillés dans le jeu, les hit-box des piétons étant des cylindres.

Jeux de données existants

Plusieurs articles présentent un jeu de données qui repart d'un jeu déjà existant, et qu'ils ont modifié et amplifié. Par exemple, Zhengqin Li et coll. [3], reprennent en partie le jeu de données Scan2CAD [5] afin d'en recréer un nouveau. Plus précisément, ils utilisent les positions initiales de Scan2CAD pour aligner les modèles CAO (conception assistée par ordinateur, soit les techniques de modélisation géométrique afin de réaliser des produits manufacturés ainsi que les outils pour les fabriquer) avec les instances de meubles. Le résultat est un jeu de données avec des images plus travaillées sur pleins d'aspects, comme la couleur, que le jeu de données Scan2CAD [5].

De la même manière, Armen Avetisyan et coll. [5] reprennent les modèles 3D des meubles des jeux de données ScanNet [4] et ShapeNet [9] pour constituer leurs scènes.

Angel X. Chang et coll. pour la création de jeu de données ShapeNet [9] ont récupéré des données provenant de dépôts publics en ligne comme Trimble 3D Warehouse1 et Yobi3D2.

Nous pouvons aussi citer l'exemple de ProcTHOR [1] qui reprend le jeu de données AI2-THOR qui est composé de 120 pièces différentes de la maison et plus de 2000 objets, avec lesquels il est possible d'interagir et de modifier leur état, créés sur Unity par des artistes en modélisation 3D. Plusieurs ajouts lui ont été faits, comme l'ajout de fenêtres, de portes, ou de comptoirs par exemple.

De la même manière, l'article "Bim-to-Scan" for Scan-to-Bim : Generating Realistic Synthetic Ground Truth Point Clouds Based on Industrial 3D Models" de Florian Noichl et coll. [18] se base sur les scènes du jeu de données HELIOS qui est un jeu comportant des données de profondeur. Florian Noichl et coll. ont simulé un laser dans l'environnement pour récupérer des données similaires à des données récupérées dans une scène d'intérieur via un capteur LIDAR, mais avec plus de précisions (moins de zones de vide que lorsqu'il s'agit de données du monde réel), et aussi en récupérant des données qu'un capteur LIDAR ne peut récupérer grâce aux annotations déjà présentes

dans la scène, comme les matériaux, le type des objets et leur sémantique par exemple. Ainsi, cela allège considérablement le temps passé pour la création des annotations.

C'est la même technique qui fût utilisée par Lukas Winiwarter et coll. pour la création du framework HELIOS++ [17], qui reprirent des données de scènes existantes, et qui ont simulé un laser virtuel pour scanner la scène afin de créer un nuage de points. Une fois la scène générée, il est possible d'inter-changer les composants de cette dernière pour en créer une nouvelle, et ainsi augmenter la taille du jeu de données. C'est une technique qui permet de générer environ 200 000 rayons par seconde, ce qui augmente considérablement la vitesse de capture des données.

Bilan de la partie

La quantité de données présentes dans un jeu de données est importante pour l'apprentissage d'un réseau de neurones en Deep Learning. Il faut une certaine quantité variée pour avoir des résultats de compréhension d'une scène qui soient cohérents avec la réalité. Cette quantité sera dépendante du temps qui sera mis, ainsi que le coût des appareils nécessaires à la récupération des données. Des appareils comme des caméras ou des capteurs LIDAR ont un coût plutôt conséquent, mais on peut aussi citer Matt Deitke et coll. (ProcTHOR [1]) qui ont dû utiliser 8 GPU NVIDIA Quadro RTX 8000 ainsi que 4 GPU NVIDIA RTX A5000 pour la génération de leur jeu de données. De plus, plus le temps de captures est long et demande de main d'œuvre, moins il y aura de données récupérées pour le jeu. Cependant, pour que l'apprentissage soit correct, il faut un maximum de scènes annotées, qui ont aussi un coût. On peut voir un résumé des informations sur le tableau 8. Nous pouvons voir avec ce tableau que les jeux de données qui ont des données générées automatiquement [2, 1] ont plus de données annotées que tous les autres.

Il faut tout de même noter qu'il est difficile de comparer la performance des différents jeux de données, tout d'abord parce qu'ils ne sont pas tous sur les mêmes types de données, et qu'il est difficile de trouver des articles comparant les différents jeux de données entre eux.

8.5.4 Annotations

Pour qu'un réseau de neurones puisse apprendre à partir des données qui lui sont fournies, il est important que ces dernières soient bien annotées. C'est pour cela que Angel X. Chang et coll. à la création de ShapeNet [9] ont entrepris de créer un jeu de données dans lequel ils ont mis un soin tout

Jeu de données	Nombre d'agence- ment de scènes	Nombre de configurations	Nombres de scènes annotées
SceneNet RGBD [2]	57	16895	5M
OpenRooms [3]	-	100 000	-
ScanNet [4]	1513	1513	2,5M
Scan2CAD [5]	3049	14 225	-
SceneNN [6]	100	100	-
ArkitScenes[7]	5047	5047	450 000
SqueezeSeg [8]	-	10 848	-
ProcTHOR [1]	18	1633	100Md
ShapeNet [9]	-	3M	220 000

TABLE 8 – Comparaison des différents jeux de données selon leur quantité de données

particulier aux annotations. En effet, il y a trois types d'annotations sur leurs scènes : les annotations géométriques qui organisent les formes par catégories de propriétés géométriques, les annotations fonctionnelles qui décrivent de fonctionnement d'objets pouvant changer d'état, et les annotations physiques qui correspondent notamment à la dimension et la densité des objets.

La plupart du temps, les annotations se font pixel par pixel [8, 12, 10, 5, 4, 3, 2], c'est-à-dire qu'à chaque pixel est attribué une ou plusieurs valeurs correspondant à l'ensemble auquel ils appartiennent (par exemple un pixel de l'ensemble canapé), ainsi que les données de profondeurs s'il y en a, et toute autre donnée qui peut être importante pour le jeu de données. Il est aussi possible de faire les annotations via des boîtes de délimitation qui englobent un objet ou une surface visible, comme avec Ryosuke Yamada et coll. [16] et Gilad Baruch et Gilad Baruch et coll. pour ArkitScenes [7].

Selon les données présentes dans le jeu de données, il y aura différent type d'annotations. Pour l'identification d'objets, les annotations auront tendance à être des boîtes qui englobent le contour des objets, tandis que pour une compréhension plus générale d'une scène, l'annotation aura plus tendance à se faire pixel par pixel.

Différentes couches d'annotations

La plupart des jeux de données ne sont constitués que d'une seule couche d'annotations, mais certains en possèdent plusieurs. Nous pouvons par exemple citer Binh-Son Hua et coll. pour SceneNN [6] qui possède des annotations fines et grossières. Les scènes du jeu de données sont recouvertes d'un maillage triangulaire qui est apposé sur 100 scènes d'intérieur. Les segments fins sont obtenus automatiquement grâce à un algorithme de segmentation utilisé sur les sommets du maillage. Cela signifie que l'étiquetage correspond aux contours d'un objet. Ensuite, pour obtenir les segments grossiers, des segments fins sont fusionnés en optimisant un champ aléatoire de Markov, soit un modèle graphique sur un ensemble de variables aléatoires qui vérifient une propriété de Markov. Une fois ces deux étapes effectuées, un utilisateur vient affiner l'étiquetage à la main. Les annotations sont aussi enrichies avec des boîtes englobantes définissant la délimitation des objets. Un résultat satisfaisant a été obtenu avec un étiquetage à deux couches, augmentant la compréhension du réseau de neurones. On peut voir un exemple d'annotations sur la figure 23.

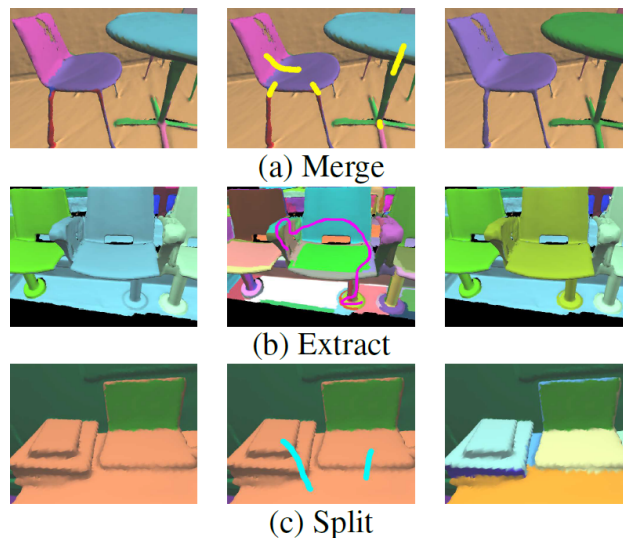


FIGURE 23 – Les images à gauche et à droite illustrent la segmentation avant et après l'application de l'opération. La fusion et la division travaillent sur les segments grossiers, tandis que l'extraction combine plusieurs segments fins en un segment grossier. Par exemple, dans (b), la chaise est extraite du sol. Image extraite de l'article "SceneNN : a Scene Meshes Dataset with aNNotations" de Binh-Son Hua et coll. [6].

Étiquetage manuel

L'étiquetage est une tâche fastidieuse qui peut très vite prendre du temps, notamment lorsqu'il s'agit de l'étiquetage par pixel. Il existe deux manières d'annoter un jeu de données, soit automatiquement, soit à la main. Nous avons pu voir au-dessus que SceneNN de Binh-Son Hua et coll. [6] a ses deux premières couches créées automatiquement via l'ordinateur, et qu'ensuite un utilisateur vient affiner le tout.

Lorsque nous faisons un étiquetage à la main, cela met beaucoup plus de temps, et plus le travail est conséquent, plus il y a un risque d'erreur humaine. De plus, selon le type de données récupérées, il peut être plus conséquent encore. Par exemple, l'annotation de données industrielles est plus longue à réaliser que celle de bureaux. Plusieurs jeux de données ont été annotés à la main, tels que celui présenté dans l'article "Accuracy Improvement of Semantic Segmentation Trained with Data Generated from a 3D Model by Histogram Matching Using Suitable References" de Miho Adachi et coll. [12] et le jeu de données Scan2CAD de Armen Avetisyan et coll. [5]. ArkitScenes de Gilad Baruch et coll. [7] a aussi été annoté à la main, mais ils ont créé un outil pour annoter les boîtes englobantes créées facilement, avec une projection en temps réel des délimitations de boîtes pour leur faciliter le travail.

Étiquetage automatique

Un étiquetage automatique se fait bien plus rapidement, et est souvent bien plus performant et moins source d'erreurs. Cependant, il faut que la scène soit assez nette et bien compréhensible pour l'ordinateur afin qu'il ne fasse pas d'erreurs, sachant qu'elles peuvent vite être conséquentes. Par exemple, l'étiquetage du jeu de données présenté par Ryosuke Yamada et coll. [16] est intégralement fait par ordinateur.

Il est difficile d'étiqueter sans avoir à passer par le Deep Learning. C'est ce qui a été fait par Ankur Handa et coll. [10] pour la segmentation sémantique. L'architecture qui a été choisie est une architecture d'encodeur-décodeur construite sur le réseau VGG. L'architecture auto-décodeur est une architecture de réseaux de neurones qui est un apprentissage non supervisé de caractéristiques discriminantes. Le réseau VGG récupère les données RGB de chaque pixel, et va affiner et diminuer la taille de l'image afin de ne conserver que les données discriminantes, ici dans l'objectif de fournir les données au réseau de neurones afin de déterminer le contour des objets et surfaces de la scène. Cependant, le réseau a été adapté afin qu'il accepte une entrée à trois canaux qui correspondent à la profondeur des pixels, la hauteur par rapport

au sol, ainsi que l'angle par rapport au sol. Une fois le contour des objets déterminé, nous utilisons leur forme afin de les identifier et les annoter. On peut voir un exemple du rendu des annotations de plusieurs exécutions sur 13 étiquettes de classe à la figure 24. On peut voir se découper les différentes formes et les objets qui en ressortent.



FIGURE 24 – La première ligne représente l'image RGB. Les lignes 2 à 4 montrent différents résultats obtenus par plusieurs exécutions. La dernière rangée montre la vérité terrain.

Image extraite de l'article "Understanding RealWorld Indoor ScenesWith Synthetic Data" de Ankur Handa et coll. [10].

Les scènes générées automatiquement ont déjà toutes les données nécessaires pour récupérer les données et annoter automatiquement, comme pour SceneNet RGBD [2] dont les données sur les matériaux, la position, la segmentation sont juste à récupérer sur chaque élément.

C'est pareil pour les scènes qui ont été capturées de manière virtuelle à partir de données déjà existantes comme Bim-to-Scan de Florian Noichl et coll. qui récupèrent, avec leur capture des profondeurs des scènes, toutes les données disponibles.

Étiquetage hybride

Une autre possibilité est de faire un étiquetage hybride, c'est-à-dire un étiquetage qui se fait par le biais de l'homme et de la machine. C'est ce

que font Angel X. Chang et coll pour ShapeNet [9] pour les annotations de ShapeNet. Dans un premier temps, un algorithme va prédire les différentes annotations sur les scènes qui lui sont fournies. Puis, une vérification est faite par plusieurs humains qui peuvent confirmer ou modifier les prédictions. Cela permet d'alléger et accélérer le travail d'annotation, tout en ayant une confirmation humaine.

Transfert d'annotations

Binh-Son Hua et coll., pour SceneNN [6], ont créé une technique pour transférer les annotations sur une autre scène similaire qui, par exemple, aurait dû être recapturée, ou dont la géométrie a légèrement changé. Dans un premier temps, il faut faire des annotations fiables aux sommets, c'est-à-dire des annotations qui dominent l'ensemble à plus de 90%. Cela signifie qu'il y a 90% de certitude, ou plus, sur l'appartenance du sommet à un ensemble. Les annotations se propageront ensuite sur les sommets n'ayant aucune annotation fiable. Il s'agit souvent de sommets en bord d'objet. Il faut noter que la source et la cible doivent partager le même ensemble de données RGB-D, c'est-à-dire qu'il faut que les différentes données proviennent d'une même scène strictement identique, ce qui limite les cas possibles de transfert. Dans la figure 25, on peut voir les trois étapes de transfert d'annotation d'une scène à une autre. Des différences subtiles de géométrie sont constatées. Les régions noires correspondent à un transfert d'annotations non fiable.

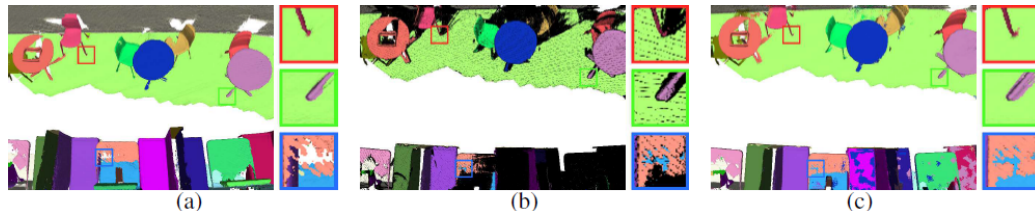


FIGURE 25 – De gauche à droite, il y a : un maillage source avec annotations, un maillage cible avec annotations transférées du maillage source et un maillage cible après propagation des annotations à l'aide de la recherche par arbre kd.

Image extraite de l'article "SceneNN : a Scene Meshes Dataset with aNNotations" de Binh-Son Hua et coll. [6].

Bilan de la partie

Le temps de création du jeu de données se fait sur le temps de récupération des données et le temps d'étiquetage. Plus l'ajout de données au jeu

Jeu de données	Méthode de capture des données	Méthode d'étiquetage
SceneNet RGBD [2]	Génération automatique	Automatique
OpenRooms [3]	Reprise jeu Scan2CAD	Automatique
ScanNet [4]	Caméras RGB-D	Automatique
Scan2CAD [5]	Caméras RGB-D	Manuelle
SceneNN [6]	Caméras RGB-D	Hybride
ArkitScenes[7]	Caméras RGB Capteurs LIDAR	Manuelle (environnement amélioré)
AdobeIndoorNav [11]	Caméras RGB-D	-
SqueezeSeg [8]	Reprise jeu KITTI Données du jeu GTA V	Automatique
ProcTHOR [1]	Génération automatique	-
ShapeNet [9]	Reprise jeux de données	Hybride

TABLE 9 – Comparaison des différents jeux de données selon leur méthode de capture et d'étiquetage

de données sera rapide et simple, plus le jeu de données deviendra rapidement conséquent. Les étiquetages manuels demandent plus de main d'œuvre et de temps de travail. Ils ont donc un coût humain plus conséquent et il faudra plus de temps pour annoter. Les annotations automatiques peuvent aisément retrouver la forme des objets, mais les différentes annotations demandent déjà une compréhension de la scène, et donc un travail assez conséquent en amont. De plus, l'humain aura tendance à faire plus régulièrement de petites erreurs, tandis que l'ordinateur fera des erreurs plus conséquentes, mais moins souvent. Une annotation hybride peut sembler, actuellement, une bonne alternative pour les scènes capturées du monde réel. Les scènes générées peuvent, elles, plus aisément annoter chaque élément de la scène. Le tableau 9 montre les différentes méthodes de capture de données, ainsi que le mode d'étiquetage lié.

8.5.5 Qualité des données

Afin que les données fournies soient utiles pour l'apprentissage, et correspondent au mieux au monde réel, il existe plusieurs variables sur lesquelles

nous pouvons jouer ; la luminosité, les différentes vues, les matériaux utilisés. Il y a aussi le bruit, qui sera toujours présent dans des captures réalisées dans le monde réel. Tous ces éléments permettent de juger, au moins en partie, de la qualité d'un jeu de données. De plus, il existe de nombreuses scènes variées, que ce soit en intérieur ou en extérieur, et selon celles qui ont été fournies, le réseau de neurones peut se retrouver perdu dans un environnement totalement différent.

Vues de la caméra

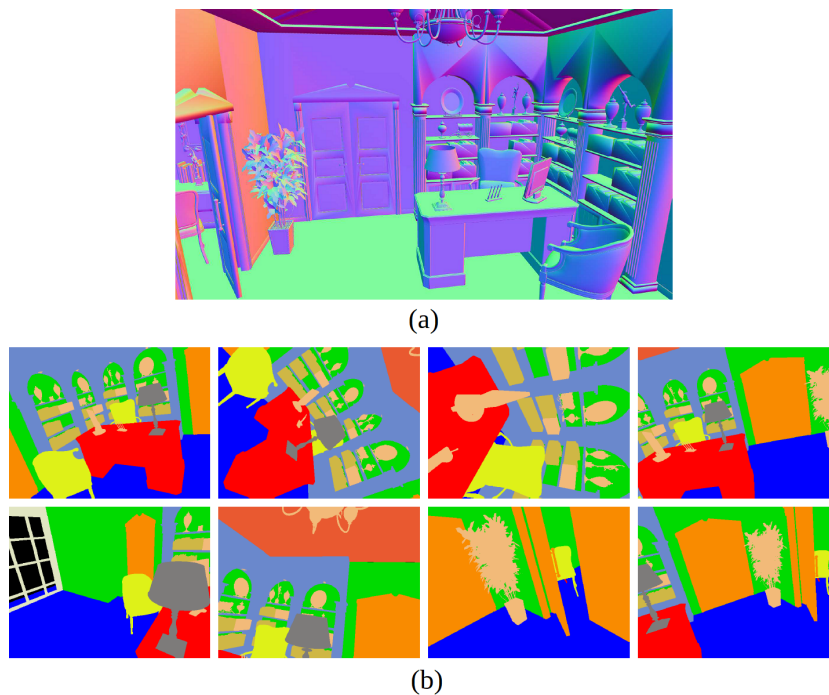


FIGURE 26 – (a) montre l'une des scènes de bureau dans SceneNet et (b) montre des images annotées rendues à partir de différents points de vue. Image extraite de l'article "Understanding RealWorld Indoor ScenesWith Synthetic Data" de Ankur Handa et coll. [10].

Lorsque les données utilisées viennent du monde réel, le nombre d'axes de vue possible est en général limité. C'est d'autant plus vrai si nous utilisons directement les données récupérées. Par exemple, l'utilisation d'un capteur de profondeur LIDAR pour capturer une scène implique que l'angle de vue et de compréhension de la scène se fera de l'endroit où le capteur a été posé initialement. S'il a été mis au sol, il sera difficile de faire le lien avec une vue du plafond. Si nous prenons l'exemple d'une voiture autonome qui sera

en permanence en mouvement, il faudra que les données fournies pour l'apprentissage correspondent à la hauteur de ses capteurs, ainsi qu'à sa manière de se déplacer. C'est le cas de Kaichun Mo et coll. [11] qui créent un jeu de données correspondant à la vision de robots qui se déplaceraient en intérieur.

Une autre solution, pour pallier ce problème, est de modéliser les données capturées en 3D, comme Ekta U. Samani et coll. [13] qui modélisent en 3D les objets afin de pouvoir les analyser sous tous leurs angles, ainsi qu'identifier des objets obstrués. De la même manière, Ankur Handa et coll. [10] modélisent les scènes initialement récupérées avec des caméras et capteurs de profondeur, puis créent des images avec un point de vue aléatoire, mais dont nous voyons toujours le plus possible de la scène, afin de permettre au réseau de neurones de s'entraîner selon tous les angles de vue. Cependant, ce travail a un coût étant donné qu'il faut, en plus de la capture initiale, transformer une image 2D en scène 3D, ce qui demande du temps, surtout s'il y a besoin d'intervention de l'homme pour la réaliser.

Binh-Son Hua et coll. pour SceneNN [6] génèrent de nouvelles images 2D avec de nouveaux points de vue, différents de ceux pris par la caméra, en passant par un modèle 3D de la même manière que cité ci-dessus.

Naturellement, c'est aussi ce qui est fait dans les jeux de données créés par ordinateur. En effet, comme les scènes sont générées en 3D avant de faire des captures en 2D, il est possible d'orienter la caméra comme on le souhaite, et de faire des captures avec de nombreux angles de vue.

On peut voir sur la figure 26 les captures sélectionnées avec une caméra aléatoire réalisées par Ankur Handa et coll. [10] pour la création de leur jeu de données, avec en (a) des exemples de scène de bureau provenant de SceneNet et en (b) des exemples de vues de la scène étiquetées sémantiquement par pixel.

Luminosité

Que ce soit en extérieur ou en intérieur, la luminosité et la position du soleil évoluent en fonction de l'heure de la journée, des saisons et de la météo dehors. De plus, en fonction des endroits par lesquels passe la lumière, il peut y avoir de forts contrastes. Par exemple, une fenêtre en plein soleil en été impactera fortement les couleurs dans la scène. Enfin, il peut y avoir d'autres sources de lumière qui impactent, que ce soit des projecteurs ou lampadaires en extérieurs, ou des lampes et écrans en tout genre en intérieur. Souvent, les lumières artificielles peuvent aussi avoir différentes intensités, ainsi que différentes couleurs. La lumière peut donc beaucoup faire évoluer l'image RGB d'une scène, là où des capteurs de profondeur ne seraient pas perturbés.

Zhengqin Li et coll. [3] ont mis beaucoup d'importance à l'ajout de lumières variées dans leur jeu de données, notamment la lumière naturelle, selon les moments de la journée, ainsi que son impact sur l'environnement, sa capacité à se réfléchir selon la surface touchée. L'objectif étant de faire la lumière la plus naturelle possible, s'adaptant selon l'heure et le temps.

Binh-Son Hua et coll. [6] utilisent une fonction de décomposition intrinsèque afin de ré-éclairer le maillage de la scène qu'ils ont récupérée. Ainsi, cela leur permet de varier la luminosité des différentes scènes du jeu de données.

Matt Deitke et coll. [1] ont, eux aussi, travaillé sur la lumière, mais celle d'intérieur. En effet, chacune de leur scène est intégralement interactive, avec la possibilité de modifier l'état de nombreux objets, dont les lumières. Ainsi, il est possible de faire varier la luminosité d'une lampe, ainsi que la couleur de son ampoule. La lumière venant de l'extérieur varie aussi selon l'heure de la journée, mais pas selon la météo.

Matériaux



FIGURE 27 – Exemples de matériaux de chaque catégorie. De gauche à droite : du tissu, du cuir, du métal, de la pierre brute, de la pierre spéculaire, du bois, de la peinture, du plastique et du caoutchouc.

Image extraite de l'article "OpenRooms : An Open Framework for Photorealistic Indoor Scene Datasets" de Zhengqin Li et coll. [3].

Lorsque les données qui sont mises dans le jeu de données sont des captures de reconstructions en 3D, ou des modélisations en 3D directement, il est assez simple de capturer la forme, mais pas les matériaux. Or, les matériaux ont un fort impact sur la scène, par exemple au niveau de la lumière. Nous pouvons facilement comprendre qu'un miroir, du cuir et du bois ne reflèteront pas la lumière de la même manière. Il faut donc que le réseau de neurones soit confronté à un maximum de matériaux différents pour pouvoir bien les appréhender. C'est principalement utile pour les jeux de données RGB.

La version la plus basique des matériaux sont de simples couleurs. C'est ce que font Binh-Son Hua et coll [6] qui se contentent d'attribuer une couleur à chaque sommet du maillage en se basant sur les couleurs de l'image initiale. C'est le même rendu que pour SceneNet [9] qui applique de simples couleurs sur ses objets 3D.

Zhengqin Li et coll. pour OpenRooms [3] ont utilisé plusieurs centaines de matériaux différents répartis dans 9 classes qui sont le tissu, le cuir, le métal, la peinture, le plastique, la pierre brute, le caoutchouc, la pierre spéculaire et le bois. Étant donné que beaucoup de données dans ShapeNet, jeu de données duquel se base OpenRooms, n'ont pas de coordonnées pour les matériaux, ils ont utilisé le mappage UV par projection cubique de Blender (un logiciel gratuit utilisé pour de la modélisation 3D créé en 2002 par Ton Roosendaal) pour calculer ces coordonnées. On peut voir un exemple des matériaux qu'ils ont créés sur la figure 27.

Matt Deitke et coll. pour ProcTHOR [1] utilisent le jeu de données AI2-THOR qu'ils ont eux-mêmes créé (<https://ai2thor.allenai.org>), qui possède déjà de nombreuses textures. Ils en ont ajouté, par exemple, les murs qui peuvent avoir soit une des 40 couleurs unies, choisies dans la liste des couleurs les plus populaires pour que le résultat ne soit pas incohérent, ou l'une des 122 textures murales comme la brique ou le carrelage. 122 matériaux AI2-THOR différents basés sur la texture ont été annotés pour qu'ils conviennent comme matériaux de mur. Il y a aussi 55 matériaux possibles pour le sol.

Bruit

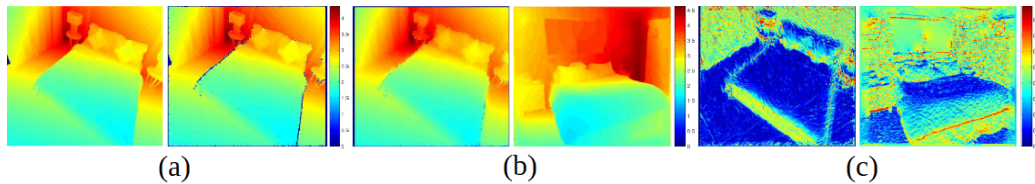


FIGURE 28 – L'image de gauche dans (a) montre la carte de profondeur parfaitement rendue et l'image de droite montre l'image bruyante correspondante fournie par le simulateur. Comparaison en (b) des cartes de profondeur peintes de scènes similaires dans SceneNet et dans les données de test NYUv2. Comparaison en (c) de l'angle avec les images de vecteur de gravité. Image extraite de l'article "Understanding RealWorld Indoor ScenesWith Synthetic Data" de Ankur Handa et coll. [10].

Les données qui sont capturées du monde réel ont systématiquement du bruit. Nous pouvons citer les capteurs de profondeur dont il manquera beaucoup de points qui leur seront obstrués par d'autres objets. Dans l'article de Miho Adachi et coll. [12], ils parlent de la difficulté à identifier le ciel, n'étant qu'un immense trou, et de l'apprentissage qu'il y a derrière pour faire comprendre au réseau de neurones ce que représente ce trou. En l'occurrence, l'objectif est de mettre le réseau de neurones dans un robot pour qu'il puisse

se déplacer en extérieur, cela fait donc partie de son apprentissage.

Il convient tout de même de reboucher les différents trous, en complétant tout d'abord avec d'autres points de vue, et en complétant ensuite par ordinateur. Nous pouvons par exemple citer Angela Dai et coll. [4] qui utilisent la reconstruction dense, qui est une technique qui détermine la géométrie 3D d'un environnement à partir des images RGB et des données de profondeurs qui ont été capturées plus tôt.

Binh-Son Hua et coll. [6] ont essayé plusieurs méthodes pour compléter les données manquantes, dont une faisant appel à l'utilisateur au moment de la segmentation pour fournir les contraintes de reconstruction 3D. Cependant, le résultat n'est pas toujours bon, les données pouvant produire alignement de surface incorrect.

À l'inverse, une fois qu'il aura appris, le réseau de neurone devra comprendre ce qui se passe devant lui, et ce, quelles que soient les données fournies. Or, lorsque les données viennent d'un modèle 3D, il n'y a aucune perte. Ankur Handa et coll. [10] ont ajouté du bruit sur leurs captures de profondeur faites sur des scènes modélisées en 3D pour être le plus proche de la réalité. Pour cela, ils ont simulé le modèle de bruit fait par Kinect.

John McCormac et coll. pour SceneNet RGBD [2] appliquent une fonction de réponse de caméra non linéaire afin de faire correspondre l'irradiance à la luminosité comme pour une vraie caméra. Il n'y a aucun bruit ou distorsion qui sont ajoutés après coup sur l'image étant donné que du bruit est naturellement ajouté lors de l'intégration des rayons de lumière. Un exemple de bruit est présent dans les données du jeu de données de Ankur Handa et coll. [10] sur la figure 28.

Types de données

Selon ce que l'on souhaite représenter, nous n'aurons pas forcément besoin de connaître l'intégralité des objets pouvant exister. Cela alourdirait inutilement les jeux de données, et complexifierait l'apprentissage. Par exemple, une voiture autonome n'a pas besoin d'identifier l'intégralité de l'électroménager d'une maison.

C'est pour cela que Angel X. Chang et coll. pour la création de ShapeNet [9] n'ont sélectionné, pour leur jeu de données sur les scènes d'intérieur, qui sont issues de dépôts publics en ligne, que les données sur les objets du quotidien, et non des modélisations 3D de pièces en mécanique, ou d'atomes par exemple.

Matt Deitke et coll. pour ProcTHOR [1] ont créé un jeu de données uniquement basé sur le domicile avec la génération de 10 000 maisons. Il y aura donc un manque si on souhaite analyser une scène dans un magasin ainsi qu'une

scène en extérieur. Il en va de même pour plusieurs autres jeux de données [3, 7, 6, 2, 10] qui sont composés de bureaux, de chambres, de cuisines et de salons.

Kaichun Mo et coll. pour AdobeIndoorNav [11] se sont plutôt axés sur les scènes dans des bureaux avec des cuisines, des espaces ouverts, des salles de conférence, de stockage et des bureaux.

Angela Dai et coll. pour ScanNet [4] ont vu plus large avec un jeu de données qui comprend des chambres, hôtels, salons, salles de séjour, salles de bains, bureaux, salles de conférence, cuisines, librairies, halls, salles de classe, appartements, allées, sous-sols, escaliers, salles à manger, buanderies, laveries et placards.

De leur côté, Ekta U. Samani et coll. [13] se sont intéressés à la modélisation d'objets afin de pouvoir les identifier dans des scènes encombrées. Ils ont donc beaucoup de petits objets dans leur jeu de données.

Enfin, Bichen Wu et coll. pour SqueezeSeg [8] et Miho Adachi et coll. [12] se sont plutôt intéressés aux scènes en extérieur, dans l'optique d'y faire déplacer un robot ou une voiture autonome.

Bilan de la partie

La qualité des données fournies par le jeu de données est importante pour un bon apprentissage. Il faut qu'elles soient suffisamment proches de la réalité pour qu'elles puissent être le plus générique possible. Dans les tableaux 10 et 11, nous pouvons voir le résumé des différents axes, explicités plus haut, représentant la qualité des différents jeux de données. Nous pouvons constater que peu de jeux de données travaillent les matériaux, la lumière et le nombre de vues, et que la majorité d'entre eux se contentent de données RGB-D.

8.5.6 Bilan

Comme dit plus haut, il est difficile de comparer l'efficacité des différents jeux de données créés, sachant que tous les jeux de données ne se concentrent pas sur les mêmes aspects, et qu'il n'y a, actuellement à ma connaissance, pas de comparaisons réalisées entre les différents jeux de données selon les catégories (scènes d'intérieurs domestiques, de bureaux, d'extérieur, d'industriel...).

De plus, on peut reprocher aux jeux de données actuels d'être principalement axés sur les scènes d'intérieur domestiques. Beaucoup de modélisations de maisons, quelques-unes de bureaux professionnels, quelques-unes de scènes d'extérieur dans la rue en ville, et quasiment aucune en industrie, ou en scène d'extérieur dans la nature. Les propositions restent donc assez limitées, mal-

Jeu de données	Type de données	Type de scène	Lumière	Matériaux
SceneNet RGBD [2]	RGB-D	Intérieur maison	Statique	Couleurs
OpenRooms [3]	RGB-D	Intérieur maison	Lumière variée	Grande quantité
ScanNet [4]	RGB-D	Tous types intérieurs	Statique	Couleurs
Scan2-CAD [5]	RGB-D Thermique	Intérieur maison	Statique	Couleurs
SceneNN [6]	RGB-D	Intérieur maison	Ré-éclairage possible	Couleurs
Arkit-Scene[7]	RGB-D	Intérieur maison	Statique	Couleurs
AdobeIndoorNav [11]	Profondeur	Intérieur maison	Statique	-
SqueezeSeg [8]	RGB	Extérieur	-	-
ProcTHOR [1]	RGB	Intérieur maison	-	Nombreux matériaux
ShapeNet [9]	RGB-D	Objets	Couleurs	Vues limitées

TABLE 10 – Comparaison des différents jeux de données selon les caractéristiques de leurs données

Jeu de données	Points de vue	Réalisme de la scène	Bruit
SceneNet RGBD [2]	Vues limitées	Peu réaliste	Aucun
OpenRooms [3]	Vues limitées	Réaliste	Aucun
ScanNet [4]	Vues limitées	Réaliste	Par obstruction
Scan2CAD [5]	Vues limitées	Réaliste	Aucun
SceneNN [6]	Vues limitées	Réaliste	Par obstruction
ArkitScenes[7]	Vues limitées	Réaliste	Par obstruction (très peu)
AdobeIndoorNav [11]	Nombreuses trajectoires	Réaliste	Réaliste
SqueezeSeg [8]	-	Réaliste	Par obstruction
ProcTHOR [1]	-	Réaliste	Aucun
ShapeNet [9]	-	Réaliste	Aucun

TABLE 11 – Comparaison des différents jeux de données selon les caractéristiques de leurs données

gré un réel besoin dans l'industrie et dans de nombreux autres domaines. De nouveaux jeux de données se concentrant sur d'autres types de scènes pourraient être les bienvenus pour permettre de développer davantage d'IA capables de s'adapter à de plus nombreuses situations.

Il en va de même pour les données proposées en sortie. La majorité des jeux de données sont RGB-D, donc principalement des images avec quelques informations sur la profondeur, et peu de nuages de points ou de données thermiques par exemple. Les applications pour l'utilisation de ces données impliquent donc d'installer des caméras sur les robots qui fonctionneront via des algorithmes entraînés sur ces jeux de données, ainsi que des capteurs LIDAR, ce qui a un certain coût et n'est pas adapté à toutes les situations. Il y a aussi parfois besoin de faire fonctionner en même temps plusieurs types de données pour permettre au réseau de neurones de s'adapter efficacement. On pourrait citer l'exemple des voitures autonomes qui ont besoin de comprendre des données de profondeurs, et peuvent avoir un plus avec la compréhension d'images RGB ou des données thermiques.

Il existe aussi un coût financier, comme les capteurs LIDAR et caméras sont souvent assez chers. Il en va de même pour les lasers qui sont plutôt onéreux, et ne sont donc pas accessibles à tous, et certainement pas au grand public. Si pour utiliser certaines applications ou jouer à certains jeux vidéos, il faut des capteurs qui ont un certain prix, la majorité des personnes n'aura pas les moyens d'accéder à cette technologie.

L'annotation est une étape primordiale et chronophage pour la création d'un jeu de données. Si elle doit être faite à la main, le temps passé dessus est d'autant plus conséquent, et est source de nombreuses petites erreurs qui peuvent gêner pour l'apprentissage. À l'inverse, des annotations automatiques ont besoin d'un algorithme suffisamment performant pour pouvoir correctement annoter les scènes. C'est une manière de faire bien plus rapide, mais dont les erreurs seront bien plus conséquentes.

Enfin, nous pouvons aussi constater que la littérature s'intéresse peu aux nuages de points, et que la question de la transformation du maillage d'une scène en nuage de points n'est pas traitée. Cette question pourrait pourtant énormément apporter lors de l'apprentissage d'un réseau de neurones.

Cependant, certains jeux de données montrent des résultats plutôt encourageants, comme Matt Deitke et coll. avec ProcTHOR [1] qui ont de très bons résultats dans la génération de scènes d'intérieur domestiques, avec de nombreux objets, matériaux, luminosités et angles de caméras possibles, et dont les scènes sont réalistes physiquement, et cohérentes. Il est donc possible en s'appliquant sur un type de données d'obtenir de bons résultats de génération, et par la même occasion, obtenir un nombre de scènes annotées assez conséquent rapidement. On peut par la suite essayer de développer d'autres

jeux de données axés sur d'autres types de scènes qui seraient tout aussi performants, et qui permettraient de sélectionner les données dont on a besoin pour l'apprentissage en Deep Learning d'une IA dans un objectif précis.

Nous pouvons aussi citer le jeu de données OpenRooms fait par Zhengqin Li et coll. [3] qui a eu un soin particulier sur la qualité et la quantité des matériaux pour les murs et les meubles, ainsi que sur la lumière selon ses différentes sources et selon l'heure de la journée pour la lumière extérieure. Cela permet d'accroître le réalisme d'une scène, et donc d'améliorer la qualité du jeu de données.

8.5.7 Conclusion

Dans cette étude bibliographique, nous avons vu plusieurs jeux de données variés pour la compréhension de scènes complexes, que ce soit en intérieur ou en extérieur.

Les points discriminants entre les jeux les plus importants que nous avons pu trouver sont la qualité des données, c'est-à-dire le travail sur l'éclairage, les matériaux, le bruit et les vues de la caméra. Il y a aussi le type d'annotations et leur nombre de couches, le type de données dont est composé le jeu, et le réalisme des scènes proposées. Nous avons pu voir de bons résultats dans la création de jeux de données performants dans la compréhension de scènes d'intérieur domestiques.

Pour la suite, il peut être intéressant d'étendre les techniques utilisées par certains bons jeux de données pour la génération de scènes à d'autres domaines, comme des scènes d'extérieur urbaines, ou des scènes industrielles. Dans le cadre de mon stage, il pourrait être intéressant de reprendre la méthode de création du jeu de données ProcTHOR, mais en l'adaptant à des données industrielles. Il serait possible de reprendre les données pour la luminosité ou les matériaux en tout genre, cependant la démarche serait différente sachant que la structure des scènes industrielles est totalement différente, étant bien plus complexe. De même, certains objets d'AI2-THOR pourront être repris, mais certains autres seront à créer, sachant que la forme de certains objets peut beaucoup varier selon leur utilisation.